C LANGUAGE

C language / 20-06-2026 • Aravinth K

GCC OPTIMIZATION FLAGS

Why Optimization Flags Exist

- By default GCC prioritizes fast compilation and easy debugging over performance. Optimization flags tell GCC — *"I'm done developing, now make the code run faster."*

All Flags

Flag	Name	What GCC Does
-O0	No optimization	Default. Exactly as you wrote. Easy to debug
-O1	Basic	Removes dead code, simple inlining
-O2	Recommended	Loop optimization, branch prediction, full inlining
-O3	Aggressive	Vectorization, unrolls loops — can cause bugs
-Os	Size optimize	Like -O2 but shrinks code size
-Og	Debug friendly	Light optimization, keeps debug info intact
-Ofast	Unsafe math	-O3 + breaks IEEE float rules

When to Use What

Flag	Situation
-O0 or -Og	During development / debugging
-O2	Production firmware / release build
-O3	Absolute max performance (risky)
-Os	Embedded — flash size is tight
-O0	Safety critical — predictable behavior

Simple Example

```
static inline int square(int x) { return x * x; }

int main() {
    int r = square(5);  // with -O0: real function call
                        // with -O2: replaced with just 5*5
}
```

INLINE FUNCTION

- A hint to the compiler to replace the function call with the function body at the call site — eliminating call overhead.

```
inline int add(int a, int b)
{
    return a + b;
}
```

Inline Without GCC Optimization

- Without -O flags, GCC ignores the inline hint and treats it like a normal function. No substitution happens.

```
gcc main.c        # inline ignored, normal call made
```

Inline With GCC Optimization

- With -O1 or higher, GCC inlines the function body at the call site.

```
gcc main.c main -O1     # inline ignored, normal call made
```

Static Inline Function

- Most common pattern. static makes the function local to the translation unit. No linker symbol emitted. Each .c file gets its own copy if needed.

```
static inline int square(int x)
{
   return x * x;
}
/* a.c → includes utils.h → gets its own copy of square()
   b.c → includes utils.h → gets its own copy of square()
   No Linker Conflict
*/
```

Safe to put in header files. No multiple-definition errors.

Inline with Forward Declaration

- Plain inline in C99 does not emit a symbol. You need an external definition somewhere, or the linker will complain.

```
// header.h
inline int mul(int a, int b)
{
     return a * b;
 }

// one .c file must have:
extern int mul(int a, int b);  // this emits the symbol
```

Extern Inline Function

- Extern inline forces symbol emission in that translation unit. Other TUs use this definition when they can't inline.

What is a "Symbol"?

- When you compile a .c file, the compiler produces a .o (object file).
  A **symbol** is just an **entry in that object file** — basically a name + address of a function or variable.

```
compile a.c → a.o  contains symbol: "mul" at address 0x100
compile b.c → b.o  contains symbol: "main" at address 0x200
```

Linker's job → **stitch all .o files together**, resolve who calls what.

Normal Function — Symbol is Always Created

```
int mul(int a, int b) { return a * b; }
```

Compile this → object file has a **real symbol mul** → linker can find it anytime.
Plain inline — NO Symbol Created

```
inline int mul(int a, int b) { return a * b; }
```

- C99 rule: "This is only for inlining. Don't create a symbol."
- So the object file has no entry for mul.

What happens if compiler skips inlining?

- Without -O2, compiler says *"I won't inline this"* → tries to call mul normally → goes to linker → linker searches all .o files → symbol mul not found anywhere → ERROR.
  - undefined reference to 'mul' ← linker error

extern Inline - Forces Symbol Creation

```
// utils.c
extern inline int mul(int a, int b) { return a * b; }
```

- This tells compiler → **"create a real symbol for mul in THIS object file."**
- Now linker can always find mul as a fallback.

Is it Like a Normal Function?

- Yes — symbol is created and globally visible. But one difference:

Flag	Tries to Inline First	Symbol Created
Normal function	❌	✅
extern inline	✅	✅ (fallback)

Advantages

	Reason
No call overhead	No push/pop of stack, no jump to function address
Faster execution	Body is right there at call site, CPU pipeline happy
Type safe	Unlike macros, compiler checks types
No side effect bugs	SQ(i++) macro bug doesn't happen
Compiler can optimize further	Compiler sees full body at call site, optimizes better

Disadvantages

	Reason
Code bloat	If called 100 times → body copied 100 times in binary
Larger binary	Bad for embedded where flash is limited
Cache pressure	Bigger binary = more instruction cache misses = **slower!**
Just a hint	Compiler can ignore inline completely
Hard to debug	No single address to put a breakpoint on
Header dependency	Body must be visible to every .c file that uses it

NESTED FUNCTIONS IN C

Nested Function

- Placing a function definition inside another function. In standard C (C89/C99/C11) this is not allowed. Functions can only be defined globally.

Standard Compiler — Not Allowed

```
#include<stdio.h>

int main()
{
    void fun()
   {                            //  ERROR: defining function inside main
        printf("GeeksForGeeks");
    }
    fun();                     // ERROR: undeclared identifier
    return 0;
}
```

```
Output:
error: function definition is not allowed here
error: use of undeclared identifier 'fun'
```

GCC Extension — Nested Functions Allowed

- Non-standard GCC extension. Not portable across compilers (MSVC, Clang strict mode will reject).

```
#include <stdio.h>

int main() {
    printf("Outer Function\n");

    void inner() {            // GCC extension – nested function definition
        printf("Inner Function\n");
    }

    inner();                  // calling nested function
    return 0;
}
```

```
Output
Outer Function
Inner Function
```

Scope & Lifetime

- Scope → nested function is visible only inside the outer function
- Lifetime → exists as long as the outer function's stack frame is alive.

Trampoline — Calling Nested Function Outside Scope

- GCC uses a trampoline — a small piece of machine code generated at runtime when you take the address of a nested function.

Trampoline holds two things:

- Real address of the nested function code
- Address of the outer function's stack frame (for variable access)
- This allows the nested function to access outer variables even when called from outside — called lexical scoping.

```c
#include <stdio.h>

void print(void (*fp)()) {    // accepts function pointer
    fp();                     // calls whatever function pointer points to
}

void outer(int x) {
    int y = 10;

    void inner() {
        printf("%d\n", x + y); // accesses x and y from outer's stack frame
    }

    void (*fp)() = &inner;    // taking address → GCC creates trampoline here
    print(fp);                // passing trampoline address to print()
}

int main() {
    outer(5);                 // x=5, y=10 → prints 15
    return 0;
}

Output :
15
```

Dangling Trampoline — Segfault Case

- If outer function returns and you call the nested function later → stack frame is destroyed → accessing x and y → segmentation fault.

```
#include <stdio.h>
void (*outer(int x))() {     // outer() returns a function pointer
    int y = 10;
    void inner() {
        printf("%d\n", x + y); // x and y live on outer's stack
    }
    return &inner;            // returning trampoline address — DANGEROUS
}
int main() {
    void (*fp)() = outer(5); // outer() finishes → stack frame destroyed
    fp();                    // inner() tries to access x,y → already gone → SEGFAULT
    return 0;
}
Output:
Segmentation fault
```

Exception — No Outer Variable Access

- If inner function **doesn't access outer variables**, calling after outer returns works fine — no stack frame dependency.

```
void inner() {
    printf("Hello from inner()"); // no x, no y — no stack dependency
}
Output:
Hello from inner()
```

Quick Summary Table

Point	Detail
Standard C	Nested functions not allowed
GCC extension	Allowed, non-portable
Scope	Only inside outer function
Lifetime	Until outer's stack frame is destroyed
Trampoline	Runtime code for calling nested fn via pointer
Lexical scoping	Inner can access outer's local variables
Danger	Never call via pointer after outer returns

ARRAYS IN C

- An array is a **linear data structure** that stores a **fixed-size** sequence of elements of the **same data type** in **contiguous memory** locations.
- Each element can be **accessed** directly **using** its **index**, which allows for efficient retrieval and modification.

```c
#include <stdio.h>

int main() {
    int arr[] = {2, 4, 8, 12, 16, 18};

    int n = sizeof(arr)/sizeof(arr[0]);

    // Printing array elements
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output :

```
2 4 8 12 16 18
```

Accessing Array Elements

```c
#include <stdio.h>

int main() {
    // array declaration and initialization
    int arr[5] = {2, 4, 8, 12, 16};

    // accessing element at index 2 i.e 3rd element
    printf("%d ", arr[2]);

    // accessing element at index 4 i.e last element
    printf("%d ", arr[4]);

    // accessing element at index 0 i.e first element
    printf("%d ", arr[0]);
    return 0;
}
```

Output :

```
8 16 2
```

Update Array Element

```
#include <stdio.h>

int main() {
    int arr[5] = {2, 4, 8, 12, 16};

    // Update the first value of the array

    arr[0] = 1;
    printf("%d", arr[0]);
    return 0;
}

Output
1
```

C Array Traversal

```
#include <stdio.h>

int main() {
    int arr[5] = {2, 4, 8, 12, 16};

    // Print each element of array using loop
    printf("Printing Array Elements\n");

    for(int i = 0; i < 5; i++){
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Printing array element in reverse
    printf("Printing Array Elements in Reverse\n");

    for(int i = 4; i>=0; i--){
        printf("%d ", arr[i]);
    }

    return 0;
}

Output :
Printing Array Elements
2 4 8 12 16
Printing Array Elements in Reverse
16 12 8 4 2
```

Size of Array

```
#include <stdio.h>
int main() {

    int arr[5] = {2, 4, 8, 12, 16};


    // Size of the array
    int size = sizeof(arr)/sizeof(arr[0]);

    printf("%d", size);

    return 0;

}
```

Output:
5

Practice Array Problems

1. How to Find Maximum Value in an Array in C?

```
/* C program to find maximum value in an array */
#include <stdio.h>

int main()
{

    // Initialize an array
    int arr[] = { 23, 12, 45, 20, 90, 89, 95, 32, 65, 19 };

    // Find the size of the array
    int n = sizeof(arr) / sizeof(arr[0]);

    // Intialize the variable which will denote the  maximum element
    int res = arr[0];

    // Find the maximum value in the array and store it in res

    for (int i = 0; i < n; i++) {
        if (res < arr[i])
            res = arr[i];
    }
    // print the elements of the array
    printf("Array Elements: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
```

```
    // print the maximum value
    printf("The maximum value of the array is: %d", res);

    return 0;
}
```

```
Output:
Array Elements: 23 12 45 20 90 89 95 32 65 19
The maximum value of the array is: 95
```

C Program to Calculate Sum of Array Elementsl

```
#include <stdio.h>

int getSum(int arr[], int n) {

    // Initialize sum to 0
    int sum = 0;
    for (int i = 0; i < n; i++) {

        // Add each element to sum
        sum += arr[i];
    }
    return sum;
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

      // Find the sum
    int res = getSum(arr, n);

    printf("%d", res);
    return 0;
}
```

```
Output
15
```

Using Recursion

```c
#include <stdio.h>

int getSum(int arr[], int n) {

    // Base case: No elements left
    if (n == 0) return 0;

    // Add current element and move to the rest of the array
    return arr[n - 1] + getSum(arr, n - 1);
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    // Finding the sum
    int res = getSum(arr, n);

    printf("%d", res);
    return 0;
}
```

```
Output:
15
```

Reverse Array in C

```c
#include <stdio.h>

void rev(int arr[], int n) {

      // Two pointers
    int l = 0, r = n - 1;
    while (l < r) {

        // Swap the elements
        int temp = arr[l];
        arr[l] = arr[r];
        arr[r] = temp;

        // Move pointers towards middle
        l++;
        r--;
    }
}
int main() {
    int arr[] = {1, 2, 3, 4, 5};
```

```
    int n = sizeof(arr) / sizeof(arr[0]);

  // Reverse array arr
    rev(arr, n);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
Output:
5 4 3 2 1
```

Using Recursion

```
#include <stdio.h>

void rev(int arr[], int l, int r) {
    if (l >= r) {
        return;
    }

    // Swap the elements
    int temp = arr[l];
    arr[l] = arr[r];
    arr[r] = temp;

    // Recursively call function to reverse the
       // remaining part
    rev(arr, l + 1, r - 1);
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

       // Reverse the array arr
    rev(arr, 0, n - 1);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
Output:
5 4 3 2 1
```

Using Temporary Array

```c
#include <stdio.h>

void rev(int arr[], int n) {
    int temp[n];

       // Store elements into temp in reverse order
    for (int i = 0; i < n; i++) {
        temp[i] = arr[n - 1 - i];
    }

    // Copy reversed array back to original array
    for (int i = 0; i < n; i++) {
        arr[i] = temp[i];
    }
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

       // Reverse the array arr
    rev(arr, n);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Output:

5 4 3 2 1

Insert Element at Specific Position

```c
#include <stdio.h>

void insert(int arr[], int *n, int pos, int val) {

    // Shift elements to the right
    for (int i = *n; i > pos; i--)
        arr[i] = arr[i - 1];

    // Insert val at the specified position
    arr[pos] = val;

    // Increase the current size
    (*n)++;
}

int main() {
    int arr[7] = {10, 20, 30, 40, 50};
```

```
    int n = 5;
    int pos = 3;
    int val = 25;


    insert(arr, &n, pos, val);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
Output
10 20 30 25 40 50
```

Insert an Element at the End of an Array

```
#include <stdio.h>

void insertLast(int arr[], int *n, int val) {

    // Insert val at last
    arr[*n] = val;

    // Increase the current size
    (*n)++;

}


int main() {

    int arr[7] = {10, 20, 30, 40, 50};

    int n = 5;

    int val = 25;

    // Insert the value at the end
    insertLast(arr, &n, val);

    for (int i = 0; i < n; i++)

        printf("%d ", arr[i]);

    return 0;

}

Output
10 20 30 40 50 25
```

C Program to Delete an Element in an Array

```
#include <stdio.h>
void del(int arr[], int *n, int key) {

       // Find the element
       int i = 0;
       while (arr[i] != key) i++;

    // Shifting the right side elements one
       // position towards left
    for (int j = i; j < *n - 1; j++) {
        arr[j] = arr[j + 1];
    }
       // Decrease the size
    (*n)--;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 30;
       // Delete the key from array
    del(arr, &n, key);

     for (int i = 0; i < n; i++)
       printf("%d ", arr[i]);

    return 0;
}
Output
10 20 40 50
```

Delete the Last Element from an Array

```
#include <stdio.h>

void delLast(int arr[], int *n) {

       // Decrease the size
    (*n)--;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);

       // Delete the last element
    delLast(arr, &n);
```

```
      for (int i = 0; i < n; i++)
      printf("%d ", arr[i]);

    return 0;
}
Output
10 20 30 40
```

C Program for Program for array rotation

```
// C++ program to illustrate how to rotate array
#include <stdio.h>


// Function to rotate array left by one position
void leftRotateByOne(int arr[], int n)

{

    int temp = arr[0];

    for (int i = 0; i < n - 1; i++) {

        arr[i] = arr[i + 1];

    }

    arr[n - 1] = temp;

}


// Function to rotate array left by d positions
void leftRotate(int arr[], int d, int n)

{

    for (int i = 0; i < d; i++) {

        leftRotateByOne(arr, n);

    }

}


// Function to print an array
void printArray(int arr[], int n)

{

    for (int i = 0; i < n; i++) {
```

```
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main()
{
    int arr[] = { 1, 2, 3, 4, 5, 6, 7 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int d = 2;

    // Rotate array left by d positions
    leftRotate(arr, d, n);

    printf("Array  after rotated by %d positions is: ", d);
    printArray(arr, n);

    return 0;
}
```

Output

```
Array  after rotated by 2 positions is: 3 4 5 6 7 1 2
```

Method 2 (A Juggling Algorithm) :

```
// C program to rotate an array by d elements
#include <stdio.h>

/* function to print an array */
void printArray(int arr[], int size);

/*Function to get gcd of a and b*/
int gcd(int a, int b);

/*Function to left rotate arr[] of size n by d*/
```

```
void leftRotate(int arr[], int d, int n)
{
    int i, j, k, temp;
    /* To handle if d >= n */
    d = d % n;
    int g_c_d = gcd(d, n);
    for (i = 0; i < g_c_d; i++) {
        /* move i-th values of blocks */
        temp = arr[i];
        j = i;
        while (1) {
            k = j + d;
            if (k >= n)
                k = k - n;
            if (k == i)
                break;
            arr[j] = arr[k];
            j = k;
        }
        arr[j] = temp;
    }
}


/*UTILITY FUNCTIONS*/
/* function to print an array */
void printArray(int arr[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
}


/*Function to get gcd of a and b*/
int gcd(int a, int b)
```

```
{
    if (b == 0)
        return a;
    else
        return gcd(b, a % b);
}


/* Driver program to test above functions */
int main()
{
    int arr[] = { 1, 2, 3, 4, 5, 6, 7 };
    leftRotate(arr, 2, 7);
    printArray(arr, 7);
    getchar();
    return 0;
}
```

Output

```
3 4 5 6 7 1 2
```

Return an Array in C

```
// C Program to return an array in C using Pointers
#include <stdio.h>
#include <stdlib.h>


// Function to create and return an array using dynamic
// memory allocation
int* createArray(int size)
{
    // Allocate memory for the array
    int* array = (int*)malloc(size * sizeof(int));


    // Check if memory allocation was successful
    if (array == NULL) {
```

```
        printf("Memory allocation failed!\n");
        exit(1); // Exit the program if allocation fails
    }

    // Initialize the array with example values
    for (int i = 0; i < size; i++) {
        array[i] = i * 2;
    }

    // Return the pointer to the allocated array
    return array;
}

int main()
{
    // Define the size of the array
    int size = 5;
    // Call the function to create the array and store the
    // returned pointer
    int* myArray = createArray(size);

    // Print the elements of the array
    printf("Array Elements: ");
    for (int i = 0; i < size; i++) {

        printf("%d ", myArray[i]);
    }
    printf("\n");

    // Free the allocated memory to avoid memory leaks
    free(myArray);
```

```
    return 0;

}
```

Output

```
Array Elements: 0 2 4 6 8
```

Return an Array in C Using Static Arrays

```
// C Program to return an array in C using static arrays
#include <stdio.h>

// Function to create and return a static array
int* createStaticArray()
{
    // Declare a static array of size 5
    static int array[5];

    // Initialize the array with example values
    for (int i = 0; i < 5; i++) {
        array[i] = i * 2;
    }

    // Return the pointer to the static array
    return array;
}

int main()
{
    // Call the function to get the static array and store
    // the returned pointer
    int* myArray = createStaticArray();

    // Print the elements of the array
    printf("Array Elements: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", myArray[i]);
    }
    printf("\n");

    return 0;
}
```

Output

```
Array Elements: 0 2 4 6 8
```

Return an Array in C Using Structures

```c
// C Program to return an array in C using Structures
#include <stdio.h>

// Define a structure to hold an array
typedef struct {
    int array[5];
} ArrayStruct;

// Function to create and return a structure containing an
// array
ArrayStruct createStructArray()
{
    // Declare a variable of type ArrayStruct
    ArrayStruct arrStruct;
    for (int i = 0; i < 5; i++) {
        // Initialize the array
        arrStruct.array[i] = i * 2;
    }
    // Return the structure
    return arrStruct;
}

int main()
{
    // Call the function to get a structure containing the
    // array
    ArrayStruct myArrayStruct = createStructArray();

    // Print the elements of the array
    printf("Array Elements: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", myArrayStruct.array[i]);
    }
    printf("\n");

    return 0;
}
```

Output

```
Array Elements: 0 2 4 6 8
```

C Program to Sort an Array in Ascending Order

```c
#include <stdio.h>
#include <stdlib.h>

// Custom comparator
int comp(const void* a, const void* b) {

      // If a is smaller, positive value will be returned
    return (*(int*)a - *(int*)b);
}

int main() {
    int arr[] = { 2 ,6, 1, 5, 3, 4 };
    int n = sizeof(arr) / sizeof(arr[0]);

    // Sort the array using qsort
    qsort(arr, n, sizeof(int), comp);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

Output

```
1 2 3 4 5 6
```

Using Selection Sort

```c
#include <stdio.h>

// Selection sort implementation
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min])
                min = j;
        }
       if (min != i) {
            int temp = arr[min];
            arr[min] = arr[i];
            arr[i] = temp;
        }
    }
}
```

```
int main() {
    int arr[] = { 2 ,6, 1, 5, 3, 4 };
    int n = sizeof(arr) / sizeof(arr[0]);

    // Perform Selection Sort
    selectionSort(arr,n);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
Output
1 2 3 4 5 6
```

Using Bubble Sort

```
#include <stdio.h>

// Bubble sort implementation
void bubbleSort(int arr[], int n) {
   for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = { 2 ,6, 1, 5, 3, 4 };
    int n = sizeof(arr) / sizeof(arr[0]);

    // Perform bubble sort
      bubbleSort(arr,n);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
Output
1 2 3 4 5 6
```

C Program to Calculate Sum of Array Elements

```c
#include <stdio.h>

int getSum(int arr[], int n) {

    // Initialize sum to 0
    int sum = 0;
    for (int i = 0; i < n; i++) {

        // Add each element to sum
        sum += arr[i];
    }
    return sum;
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

      // Find the sum
    int res = getSum(arr, n);

    printf("%d", res);
    return 0;
}
```

Output

15

Using Recursion

```c
#include <stdio.h>

int getSum(int arr[], int n) {

    // Base case: No elements left
    if (n == 0) return 0;

    // Add current element and move to the
      // rest of the array
    return arr[n - 1] + getSum(arr, n - 1);
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
```

```
    // Finding the sum
    int res = getSum(arr, n);

    printf("%d", res);
    return 0;
}
Output
15
```

MULTIDIMENSIONAL ARRAYS IN C - 2D AND 3D ARRAYS

2D Array

- Think of it as table rows and columns.

```
int arr[3][4];   // 3 rows, 4 columns
```

Declaration & Initialization

```
int arr[3][4] = {
    {1,  2,  3,  4},   // row 0
    {5,  6,  7,  8},   // row 1
    {9, 10, 11, 12}    // row 2
};
```

How it looks logically

```
col0  col1  col2  col3
row0 [  1     2     3     4  ]
row1 [  5     6     7     8  ]
row2 [  9    10    11    12  ]
```

How it is stored in memory (Row-Major Order)

C stores 2D arrays row by row in a single continuous block.

```
Address:  100  104  108  112  116  120  124  128  132  136  140  144
Value:      1    2    3    4    5    6    7    8    9   10   11   12
           |--- row0 ---|--- row1 ---|--- row2 ---|
```

Each int = 4 bytes. All rows placed one after another.

3D Array

- Think of it as multiple tables — layers of 2D arrays.

```
int arr[2][3][4];   // 2 layers, 3 rows, 4 columns
```

Declaration & Initialization

```
int arr[2][3][4] = {
```

```
    {                       // layer 0
        {1,  2,  3,  4},  // layer0, row0
        {5,  6,  7,  8},  // layer0, row1
        {9, 10, 11, 12}   // layer0, row2
    },
    {                        // layer 1
        {13, 14, 15, 16},  // layer1, row0
        {17, 18, 19, 20},  // layer1, row1
        {21, 22, 23, 24}   // layer1, row2
    }
};
```

How it is stored in memory

All layers, row by row, in one continuous block.

```
Layer 0:
 1   2   3   4   5   6   7   8   9  10  11  12
|--- row0 ---|--- row1 ---|--- row2 ---|

Layer 1:
13  14  15  16  17  18  19  20  21  22  23  24
|--- row0 ---|--- row1 ---|--- row2 ---|
```

Full memory block:

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 | 13 14 15 16 17 18 19 20 21 22 23 24 ]
|-------------- layer 0 -------------|--------------- layer 1 ---------------|
```

simple code to initialize, overwrite existing value and read the modified value example in C

```c
#include <stdio.h>

int main() {

    /* ====================== 2D ARRAY ====================== */

    /* Initialize a 2D array: 3 rows, 4 columns */
    int arr2d[3][4] = {
        {1,  2,  3,  4},
        {5,  6,  7,  8},
        {9, 10, 11, 12}
    };

    /* Print original values */
```

```
    printf("2D - Before Modification:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            printf("%4d", arr2d[i][j]);   /* %4d for aligned output */
        }
        printf("\n");
    }

    /* Overwrite all values using nested loop */
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            arr2d[i][j] = arr2d[i][j] * 2;   /* multiply every element by 2 */
        }
    }

    /* Print modified values */
    printf("\n2D - After Modification (each value x2):\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            printf("%4d", arr2d[i][j]);
        }
        printf("\n");
    }


    /* ====================== 3D ARRAY ====================== */

    /* Initialize a 3D array: 2 layers, 3 rows, 4 columns */
    int arr3d[2][3][4] = {
        {
            {1,  2,  3,  4},
            {5,  6,  7,  8},
            {9, 10, 11, 12}
        },
        {
            {13, 14, 15, 16},
            {17, 18, 19, 20},
            {21, 22, 23, 24}
        }
    };

    /* Print original values */
    printf("\n3D - Before Modification:\n");
    for (int i = 0; i < 2; i++) {            /* loop layers */
        printf("Layer %d:\n", i);
        for (int j = 0; j < 3; j++) {        /* loop rows */
            for (int k = 0; k < 4; k++) {    /* loop columns */
                printf("%4d", arr3d[i][j][k]);
            }
            printf("\n");
        }
```

```
    }

    /* Overwrite all values using triple nested loop */
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            for (int k = 0; k < 4; k++) {
                arr3d[i][j][k] = arr3d[i][j][k] * 2;   /* multiply every element
by 2 */
            }
        }
    }

    /* Print modified values */
    printf("\n3D - After Modification (each value x2):\n");
    for (int i = 0; i < 2; i++) {
        printf("Layer %d:\n", i);
        for (int j = 0; j < 3; j++) {
            for (int k = 0; k < 4; k++) {
                printf("%4d", arr3d[i][j][k]);
            }
            printf("\n");
        }
    }

    return 0;
}
```

Output

```
2D - Before Modification:

   1   2   3   4

   5   6   7   8

   9  10  11  12


2D - After Modification (each value x2):

   2   4   6   8

  10  12  14  16

  18  20  22  24


3D - Before Modification:

Layer 0:

   1   2   3   4

   5   6   7   8
```

```
   9  10  11  12

Layer 1:

  13  14  15  16

  17  18  19  20

  21  22  23  24


3D - After Modification (each value x2):

Layer 0:

   2   4   6   8

  10  12  14  16

  18  20  22  24

Layer 1:

  26  28  30  32

  34  36  38  40

  42  44  46  48
```

C program to pass a 3D array to a function

```c
// C program to pass a 3D array to a function

#include <stdio.h>


// Function to print the elements of a 3D array
void printArray(int arr[][3][3], int rows, int cols,
                int depth)
{
    printf("Elements of the 3D array:\n");

    // Loop through each element in the 3D array
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            for (int k = 0; k < depth; ++k) {
                // Print each element
                printf("%d   ", arr[i][j][k]);
            }
```

```
            // Printing a new line at the end of each column
            printf("\n");

        }

        // Printing a new line at the end of each row
        printf("\n");

    }

}


int main()

{

    // Initialize the 3D array with fixed sizes
    int arr[3][3][3] = {

        { { 10, 20, 30 }, { 40, 50, 60 }, { 70, 80, 90 } },

        { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } },

        { { 190, 200, 210 },

          { 220, 230, 240 },

          { 250, 260, 270 } }

    };


    // Declare the dimensions of the array
    int rows = 3;

    int cols = 3;

    int depth = 3;


    // Pass the 3D array to the function
    printArray(arr, rows, cols, depth);


    return 0; // Return 0 to indicate successful execution

}
```

Output

```
Elements of the 3D array:
10   20   30
40   50   60
70   80   90

1   2   3
4   5   6
```

```
7   8   9

190   200   210
220   230   240
250   260   270
```

STRINGS IN C

- A string in C is an array of characters terminated by a null character '\0'.
- The null character '\0' marks the end of the string.
- C does not have a built-in string data type.
- Strings are implemented using arrays of char.

Declaration & Initialization

```
#include <stdio.h>

int main() {

    /* Declaring and initializing a string
       compiler automatically appends '\0' at the end */
    char str[] = "Embedded";

    /* %s prints characters starting from str[0]
       until it hits the null character '\0' */
    printf("The string is: %s\n", str);

    return 0;
}
Output:
The string is: Embedded
```

char str[] = "Embedded" internally creates:

{ 'E', 'm', 'b', 'e', 'd', 'd', 'e', 'd', '\0' }

The '\0' is automatically added at the end. %s prints each character until it finds '\0'.

Accessing Characters

Access any character using its index, just like an array.

```
#include <stdio.h>

int main() {

    char str[] = "Embedded";

    /* str[0] accesses the first character 'E'
```

```
       index starts from 0 */
    printf("%c\n", str[0]);   /* prints E */
    printf("%c\n", str[4]);   /* prints 'd' — 5th character */

    return 0;
}
Output
E
d
```

Update / Modify

Individual characters can be changed using index. For replacing the full string, use strcpy(). Always ensure the new string fits within the allocated array size.

```
#include <stdio.h>

int main() {

    char str[] = "Embedded";

    /* Modify the first character from 'E' to 'X'
       direct index assignment changes only that one character */
    str[0] = 'X';

    printf("%s\n", str);   /* prints Xmbedded */

    return 0;
}
Output
Xmbedded
```

String Length

strlen() counts characters from index 0 until it hits '\0'. The null character itself is not counted.

```
#include <stdio.h>
#include <string.h>   /* required for strlen() */

int main() {

    char str[] = "Embedded";

    /* strlen() walks the array and counts characters
       until '\0' — does not count '\0' itself */
    printf("Length: %d\n", strlen(str));   /* prints 8 */
```

```
    return 0;
}
Output
Length: 8
Note :
#include <string.h> is required to use strlen(). Without it, you get a compiler
warning or error.
```

String Input

Using scanf()

Simplest method. Limitation: stops reading at whitespace (space, tab, newline).

```
#include <stdio.h>

int main() {

    char str[10];

    /* scanf with %s reads until it hits a space or newline
       so "Embedded System" would only store "Embedded" */
    scanf("%s", str);

    printf("%s\n", str);

    return 0;
}
Input:
Embedded
Output:
Embedded
```

Using scanf() with Scanset

Scanset allows specifying which characters to include or exclude. [^\n] means read everything except newline — so spaces are included.

```
#include <stdio.h>

int main() {

    char str[20];

    /* %19[^\n] reads up to 19 characters
       stops only at newline '\n'
       19 not 20 — leaves 1 byte for '\0' */
```

```
    scanf("%19[^\n]", str);

    printf("%s\n", str);

    return 0;
}
Input:
Embedded System
Output:
Embedded System
```

Using fgets()

Recommended for reading strings with spaces. Reads the entire line including spaces until newline.

```
#include <stdio.h>

int main() {

    char str[20];

    /* fgets(buffer, size, stream)
       reads at most 19 characters (size-1) from stdin
       automatically adds '\0' at end
       also stores the newline '\n' if it fits in buffer */
    fgets(str, 20, stdin);

    printf("%s\n", str);

    return 0;
}
Input:
Embedded System
Output:
Embedded System
Note :
fgets() stores the newline '\n' in the buffer if there is space. This can cause an
extra blank line when printing. To remove it:
```

Passing Strings to a Function

Strings (character arrays) are passed as a pointer to the first character. No copy is made — the function works on the original array.

```
#include <stdio.h>

/* str[] here is actually treated as char* by the compiler
```

```
   both char str[] and char *str work as parameter */
void printStr(char str[]) {

    printf("%s\n", str);

}

int main() {

    char str[] = "Embedded System";

    /* passing string to function – passes address of str[0] */
    printStr(str);

    return 0;

}

Output
Embedded System
```

Strings and Pointers

A character pointer can point to the starting address of a string. Incrementing the pointer moves it to the next character.

```
#include <stdio.h>

int main() {

    char str[20] = "Embedded";

    /* ptr stores the address of str[0] – first character 'E' */
    char *ptr = str;

    /* dereference ptr to get the character
       ptr++ moves to the next character each iteration
       loop stops when '\0' is reached */
    while (*ptr != '\0') {
        printf("%c", *ptr);
        ptr++;
    }
    printf("\n");

    return 0;
}
Output:
Embedded
```

String Literals

A string literal is a sequence of characters in double quotes like "Embedded System". Stored in **read-only memory** by the compiler.

```
#include <stdio.h>

int main() {

    /* const char* points to a string literal in read-only memory
       const is important — prevents accidental modification
       modifying a string literal causes undefined behavior */
    const char *str = "Embedded System";

    /* printf starts at address in str
       prints until '\0' */
    printf("%s\n", str);

    return 0;
}
Output
Embedded System
```

Key points:

- String literals are automatically null terminated.
- Stored in read-only memory — modifying causes undefined behavior.
- Always use const char* when pointing to a string literal, not plain char*.

Difference between char[] and char* for strings:

```
char str[] = "Embedded";    /* copy on stack — safe to modify */
str[0] = 'X';               /* works fine */

const char *ptr = "Embedded"; /* points to read-only memory */
ptr[0] = 'X';                 /* undefined behavior — crash */
```

C STRING FUNCTIONS

- C language provides various built-in functions that can be used for various operations and manipulations on strings. These string functions make it easier to perform tasks such as string copy, concatenation, comparison, length, etc. The <string.h> header file contains these string functions.

strlen()

- The strlen() function is used to find the length of a string. It returns the number of characters in a string, excluding the null terminator ('\0').

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[] = "Gfg";

      // Finding and printing length of string s
    printf("%lu", strlen(s));
    return 0;
}
Output
3
```

strcpy()

- The strcpy() function copies a string from the source to the destination. It copies the entire string, including the null terminator.

```
#include <stdio.h>
#include <string.h>

int main() {
    char src[] = "Hello";
    char dest[20];

    // Copies "Hello" to dest
    strcpy(dest, src);
    printf("%s", dest);
    return 0;
}
Output
Hello
```

strncpy()

- The strncpy() function is similar to strcpy(), but it copies at most n bytes from source to destination string. If source is shorter than n, strncpy() adds a null character to destination to ensure n characters are written.

```
#include <stdio.h>
#include <string.h>

int main() {
    char src[] = "Hello";
    char dest[20];

    // Copies "Hello" to dest
    strncpy(dest, src, 4);
    printf("%s", dest);
    return 0;
}
Output
Hell
```

strcat()

- The **strcat()** function is used to concatenate (append) one string to the end of another. It appends the source string to the destination string, replacing the null terminator of the destination with the source string's content.

```
#include <stdio.h>
#include <string.h>

int main() {
    char s1[30] = "Hello, ";
    char s2[] = "Geeks!";

    // Appends "Geeks!" to "Hello, "
    strcat(s1, s2);
    printf("%s", s1);
    return 0;
}
Output
Hello, Geeks!
```

strncat()

- In C, there is a function strncat() similar to strcat(). This function appends not more than n characters from the string pointed to by source to the end of the string pointed to by destination plus a terminating NULL character.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s1[30] = "Hello, ";
    char s2[] = "Geeks!";

    // Appends "Geeks!" to "Hello, "
    strncat(s1, s2, 4);
    printf("%s", s1);
    return 0;
}
```

Output

```
Hello, Geek
```

strcmp()

- The strcmp() is a built-in library function in C. This function takes two strings as arguments, compares these two strings lexicographically and returns an integer value as a result of comparison.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s1[] = "Apple";
    char s2[] = "Applet";

      // Compare two strings
      // and print result
    int res = strcmp(s1, s2);
    if (res == 0)
        printf("s1 and s2 are same");
      else if (res < 0)
      printf("s1 is lexicographically "
             "smaller than s2");
      else
      printf("s1 is lexicographically "
            "greater than s2");
    return 0;
}
```

Output

```
s1 is lexicographically smaller than s2
```

strncmp()

- This function lexicographically compares the first n characters from the two null-terminated strings and returns an integer based on the outcome.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s1[] = "Apple";
    char s2[] = "Applet";

      // Compare two strings upto
      // 4 characters and print result
    int res = strncmp(s1, s2, 4);
    if (res == 0)
        printf("s1 and s2 are same");
      else if (res < 0)
      printf("s1 is lexicographically "
             "smaller than s2");
      else
      printf("s1 is lexicographically "
      "greater than s2");
    return 0;
}
```

```
Output
s1 and s2 are same
```

strchr()

- The strchr() function is used to find the first occurrence of a given character in a string. If the character is found, it returns a pointer to the first occurrence of the character; otherwise, it returns NULL.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[] = "Hello, World!";

      // Finding the first occurence of 'o' in string s
    char *res = strchr(s, 'o');
    if (res != NULL)
        printf("Character found at: %ld index", res - s);
    else
        printf("Character not found");
    return 0;
}
```

```
Output
Character found at: 4 index
```

strrchr()

- In C, **strrchr()** function is similar to strchr() function used to find the last occurrence of a given character in a string.

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[] = "Hello, World!";

      // Finding the last occurence of 'o' is string s
    char *res = strrchr(s, 'o');

    if (res != NULL)
        printf("Character found at: %ld index", res - s);
    else
        printf("Character not found\n");
    return 0;
}
Output
Character found at: 8 index
```

strstr()

- The **strstr(**) function in C is used to search the first occurrence of a substring in another string. If it is not found, it returns a NULL.

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[] = "Hello, Geeks!";

      // Find the occurence of "Geeks" in string s
    char *pos = strstr(s, "Geeks");

    if (pos != NULL)
        printf("Found");
    else
        printf("Not Found");
    return 0;
}
Output
Found
```

sprintf()

- The sprintf() function is used to format a string and store it in a buffer. It is similar to printf(), but instead of printing the result, it stores it in a string.

```
#include <stdio.h>

int main() {
    char s[50];
    int n = 10;

    // Output formatted string into string bugger s
    sprintf(s, "The value is %d", n);
    printf("%s", s);
    return 0;
}
Output
The value is 10
```

strtok()

- The strtok() function is used to split a string into tokens based on specified delimiters. It modifies the original string by replacing delimiters with null characters ('\0').

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[] = "Hello, Geeks, C!";

      // Initializing tokens
    char *t = strtok(s, ", ");

      // Printing rest of the tokens
    while (t != NULL) {
        printf("%s\n", t);
        t = strtok(NULL, ", ");
    }
    return 0;
}
Output
Hello
Geeks
C!
```

The below table lists some of the most commonly used string functions in the C language:

Function	Description	Syntax
strlen()	Find the length of a string excluding '\0' NULL character.	**strlen**(str);
strcpy()	Copies a string from the source to the destination.	**strcpy**(dest, src);
strncpy()	Copies n characters from source to the destination.	**strncpy**(dest, src, n);
strcat()	Concatenate one string to the end of another.	**strcat**(dest, src);
strncat()	Concatenate n characters from the string pointed to by src to the end of the string pointed to by dest.	**strncat**(dest, src, n);
strcmp()	Compares these two strings lexicographically.	**strcmp**(s1, s2);
strncmp()	Compares first n characters from the two strings lexicographically.	**strncmp**(s1, s2, n);
strchr()	Find the first occurrence of a character in a string.	**strchr(s, c);**
strrchr()	Find the last occurrence of a character in a string.	**strchr(*s*, *ch*);**
strstr()	First occurrence of a substring in another string.	**strstr(s, subS);**
sprintf()	Format a string and store it in a string buffer.	**sprintf(s, format, ...);**
strtok()	Split a string into tokens based on specified delimiters.	**strtok(s, delim);**

POINTERS IN C

A pointer stores the memory address of another variable instead of a direct value.

- Declared with * before the name → data_type *pointer_name;
- The data type tells what kind of variable it points to → int *ptr; points to an integer.
- Using the pointer directly gives the stored address. To get the actual value at that address, use * again — this is called **dereferencing**.
- '*'as two roles in pointers — declaring a pointer variable, and dereferencing to read the value at the stored address.

```
#include <stdio.h>

int main()
{

    // Normal Variable
    int var = 10;

    // Pointer Variable ptr that stores address of var
    int *ptr = &var;

    // Directly accessing ptr will give us an address
    printf("%p", ptr);

    return 0;
}
Output
0x7fffffffe9cc
```

Initialize a Pointer

- Assign the address of a variable using & operator → pointer_name = &variable;
- Always initialize before use — uninitialized pointers point to garbage addresses.
- If not pointing to anything yet, set to NULL → int *ptr = NULL;

Deference a Pointer

- Use * on the pointer to read or modify the value at the stored address → *ptr
- Same * symbol used in declaration, but different purpose here — this one reads the value.

```
#include <stdio.h>

int main() {
    int var = 10;
```

```
    // Store address of var variable
    int* ptr = &var;

    // Dereferencing ptr to access the value
    printf("%d", *ptr);

    return 0;
}
Output
10
```

Size of Pointers

- The size of a pointer in C depends on the architecture (bit system) of the machine, not the data type it points to.
  - On a 32-bit system, all pointers typically occupy 4 bytes.
  - On a 64-bit system, all pointers typically occupy 8 bytes.
- The size remains constant regardless of the data type (int*, char*, float*, etc.). We can verify this using the

```
#include <stdio.h>

int main() {
    int *ptr1;
    char *ptr2;

    // Finding size using sizeof()
    printf("%zu\n", sizeof(ptr1));
    printf("%zu", sizeof(ptr2));

    return 0;
}
Output
8
8
```

Special Types of Pointers

There are 4 special types of pointers that used or referred to in different contexts:

NULL Pointer

- The NULL Pointers are those pointers that do not point to any memory location.
- They can be created by assigning NULL value to the pointer. A pointer of any type can be assigned the NULL value.
- This allows us to check whether the pointer is pointing to any valid memory location by checking if it is equal to NULL.

```c
#include <stdio.h>

int main() {
    // Null pointer
    int *ptr = NULL;

    return 0;
}
```

Void Pointer

- The void pointers in C are the pointers of type void.
- It means that they do not have any associated data type.
- They are also called generic pointers as they can point to any type and can be typecasted to any type.

```c
#include <stdio.h>

int main() {
    // Void pointer
    void *ptr;

    return 0;
}
```

Wild Pointers

- The wild pointers are pointers that have not been initialized with something yet. These types of C-pointers can cause problems in our programs and can eventually cause them to crash. If values are updated using wild pointers, they could cause data abort or data corruption.

Dangling Pointer

- A pointer pointing to a memory location that has been deleted (or freed) is called a dangling pointer.
- Such a situation can lead to unexpected behavior in the program and also serve as a source of bugs in programs C.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int* ptr = (int*)malloc(sizeof(int));

    // After below free call, ptr becomes a dangling pointer
```

```
    free(ptr);
    printf("Memory freed\n");

    // removing Dangling Pointer
    ptr = NULL;

    return 0;
}
Output
Memory freed
```

Pointer Arithmetic

- Pointer Arithmetic is the set of valid arithmetic operations that can be performed on pointers. The pointer variables store the memory address of another variable. It doesn't store any value.
- Hence, there are only a few operations that are allowed to perform on Pointers in C language. The C pointer arithmetic operations are slightly different from the ones that we generally use for mathematical calculations. These operations are:
    1. Increment/Decrement of a Pointer
    2. Addition of integer to a pointer
    3. Subtraction of integer to a pointer
    4. Subtracting two pointers of the same type
    5. Comparison of pointers

Note: When we use pointers in programs, their addresses keep changing on every run because modern operating systems use ASLR(Address space layout randomization), which randomizes memory locations for security.

1. Increment/Decrement of a Pointer

- ptr++ moves the pointer **forward** by the size of its data type, not by 1 byte.
- ptr-- moves the pointer **backward** by the size of its data type.

Pointer type	Size	ptr++ from 1000	ptr-- from 1000
int *	4 bytes	1004	996
float *	4 bytes	1004	996
char *	1 byte	1001	999
double *	8 bytes	1008	992

```
#include <stdio.h>
// pointer increment and decrement
// pointers are incremented and decremented
// by the size of the data type they point to
int main(){
    int a = 22;
    int *p = &a;

    // p = 6422288
```

```
    printf("p = %u\n", p);
    p++;
    // p++ = 6422292    +4
    // 4 bytes
    printf("p++ = %u\n", p);
    p--;

    // p-- = 6422288     -4
    // restored to original value
    printf("p-- = %u\n", p);
    float b = 22.22;
    float *q = &b;

    // q = 6422284
    printf("q = %u\n", q);
    q++;

    // q++ = 6422288      +4
    // 4 bytes
    printf("q++ = %u\n", q);
    q--;

    // q-- = 6422284       -4
    // restored to original value
    printf("q-- = %u\n", q);

    char c = 'a';
    char *r = &c;

    // r = 6422283
    printf("r = %u\n", r);
    r++;

    // r++ = 6422284     +1
    // 1 byte
    printf("r++ = %u\n", r);
    r--;

    //r-- = 6422283     -1
    // restored to original value
    printf("r-- = %u\n", r);

    return 0;
}
Output
p = 1441900792
p++ = 1441900796
p-- = 1441900792
q = 1441900796
q++ = 1441900800
q-- = 1441900796
```

```
r = 1441900791
r++ = 1441900792
r-- = 1441900791
```

2. Addition of Integer to Pointer

- When a pointer is added with an integer value, the value is first multiplied by the size of the data type and then added to the pointer.

For Example:

Consider the same example as above where the ptr is an integer pointer that stores 1000 as an address. If we add integer 5 to it using the expression, ptr = ptr + 5, then, the final address stored in the ptr will be ptr = 1000 + sizeof(int) * 5 = 1020.

```
#include <stdio.h>

int main(){
    // Integer variable
    int N = 4;

    // Pointer to an integer
    int *ptr1, *ptr2;

    // Pointer stores the address of N
    ptr1 = &N;
    ptr2 = &N;

    printf("Pointer ptr2 before Addition: ");
    printf("%p \n", ptr2);

    // Addition of 3 to ptr2
    ptr2 = ptr2 + 3;
    printf("Pointer ptr2 after Addition: ");
    printf("%p \n", ptr2);

    return 0;
}
Output
Pointer ptr2 before Addition: 0x7ffca373da9c
Pointer ptr2 after Addition: 0x7ffca373daa8
```

3. Subtraction of Integer to Pointer

- When a pointer is subtracted with an integer value, the value is first multiplied by the size of the data type and then subtracted from the pointer similar to addition.

For Example:

Consider the same example as above where the ptr is an integer pointer that stores 1000 as an address. If we subtract integer 5 from it using the expression, ptr = ptr - 5, then, the final address stored in the ptr will be ptr = 1000 - sizeof(int) * 5 = 980.

```c
#include <stdio.h>

int main(){
    // Integer variable
    int N = 4;

    // Pointer to an integer
    int *ptr1, *ptr2;

    // Pointer stores the address of N
    ptr1 = &N;
    ptr2 = &N;

    printf("Pointer ptr2 before Subtraction: ");
    printf("%p \n", ptr2);

    // Subtraction of 3 to ptr2
    ptr2 = ptr2 - 3;
    printf("Pointer ptr2 after Subtraction: ");
    printf("%p \n", ptr2);

    return 0;
}
```

```
Output
Pointer ptr2 before Subtraction: 0x7ffd718ffebc
Pointer ptr2 after Subtraction: 0x7ffd718ffeb0
```

4. Subtraction of Two Pointers

- The subtraction of two pointers is possible only when they have the same data type. The result is generated by calculating the difference between the addresses of the two pointers and calculating how many bytes of data it is according to the pointer data type. The subtraction of two pointers gives the increments between the two pointers.

For Example:

Two integer pointers say ptr1(address:1000) and ptr2(address:1004) are subtracted. The difference between addresses is 4 bytes. Since the size of int is 4 bytes, therefore the increment between ptr1 and ptr2 is given by (4/4) = 1.

```c
#include <stdio.h>

int main(){
    int x = 6;
    int N = 4;

    // Pointer declaration
    int *ptr1, *ptr2;
```

```
    // stores address of N
    ptr1 = &N;

    // stores address of x
    ptr2 = &x;

    printf(" ptr1 = %u, ptr2 = %u\n", ptr1, ptr2);
    // %p gives an hexa-decimal value,
    // We convert it into an
    // unsigned int value by using %u

    // Subtraction of ptr2 and ptr1
    x = ptr1 - ptr2;

    // Print x to get the Increment
    // between ptr1 and ptr2
    printf("Subtraction of ptr1 "
           "& ptr2 is %d\n",
           x);

    return 0;
}
Output
ptr1 = 2715594428, ptr2 = 2715594424
Subtraction of ptr1 & ptr2 is 1
```

5. Comparison of Pointers

- We can compare the two pointers by using the comparison operators in C. We can implement this by using all operators in C **>, >=, <, <=, ==, !=.** It returns true for the valid condition and returns false for the unsatisfied condition.

Step 1: Initialize the integer values and point these integer values to the pointer.
Step 2: Now, check the condition by using comparison or relational operators on pointer variables.
Step 3: Display the output.

```
#include <stdio.h>

int main(){
    // declaring array
    int arr[5];

    // declaring pointer to array name
    int* ptr1 = &arr;
    // declaring pointer to first element
    int* ptr2 = &arr[0];
```

```
    if (ptr1 == ptr2) {
        printf("Pointer to Array Name and First Element "
               "are Equal.");
    }
    else {
        printf("Pointer to Array Name and First Element "
               "are not Equal.");
    }

    return 0;
}

Output
Pointer to Array Name and First Element are Equal.
```

Comparison to NULL

- In the below approach, it results in the count of odd numbers and even numbers in an array. We are going to implement this by using a pointer.

  Step 1: First, declare the length of an array and array elements.
  Step 2: Declare the pointer variable and point it to the first element of an array.
  Step 3: Initialize the count_even and count_odd. Iterate the for loop and check the conditions for the number of odd elements and even elements in an array
  Step 4: Increment the pointer location ptr++ to the next element in an array for further iteration.
  Step 5: Print the result.

```
#include <stdio.h>

int main(){
    int n = 10;

    int arr[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

    // Declaration of pointer variable
    int* ptr;

    // Pointer points the first (0th index)
    // element in an array
    ptr = arr;
    int count_even = 0;
    int count_odd = 0;

    for (int i = 0; i < n; i++) {

        if (*ptr % 2 == 0) {
            count_even++;
```

```
        }
        if (*ptr % 2 != 0) {
            count_odd++;
        }

        // Pointing to the next
        // element in an array
        ptr++;
    }
    printf("No of even elements in an array is : %d",
           count_even);
    printf("\nNo of odd elements in an array is : %d",
           count_odd);
}
```

Output

```
No of even elements in an array is : 5

No of odd elements in an array is : 5
```

Pointer Arithmetic on Arrays

- Pointers contain addresses. Adding two addresses makes no sense because there is no idea what it would point to. Subtracting two addresses lets you compute the offset between the two addresses. An array name acts like a pointer constant. The value of this pointer constant is the address of the first element.

For Example: if an array is named arr then arr and &arr[0] can be used to reference the array as a pointer.

Below is the program to illustrate the Pointer Arithmetic on arrays:

```
#include <stdio.h>

int main(){

    int N = 5;

    // An array
    int arr[] = { 1, 2, 3, 4, 5 };

    // Declare pointer variable
    int* ptr;

    // Point the pointer to first
    // element in array arr[]
    ptr = arr;

    // Traverse array using ptr
    for (int i = 0; i < N; i++) {
```

```
        // Print element at which
        // ptr points
        printf("%d ", ptr[0]);
        ptr++;
    }
}
Output
1 2 3 4 5
```

Program 2:

```
#include <stdio.h>

// Function to traverse 2D array
// using pointers
void traverseArr(int* arr, int N, int M){

    int i, j;

    // Traverse rows of 2D matrix
    for (i = 0; i < N; i++) {

        // Traverse columns of 2D matrix
        for (j = 0; j < M; j++) {

            // Print the element
            printf("%d ", *((arr + i * M) + j));
        }
        printf("\n");
    }
}

int main(){

    int N = 3, M = 2;

    // A 2D array
    int arr[][2] = { { 1, 2 }, { 3, 4 }, { 5, 6 } };

    // Function Call
    traverseArr((int*)arr, N, M);
    return 0;
}

Output

1 2
3 4
5 6
```

Constant Pointers

- In constant pointers, the memory address stored inside the pointer is constant and cannot be modified once it is defined. It will always point to the same memory address.

```
#include <stdio.h>

int main() {
    int a = 90;
    int b = 50;

    // Creating a constant pointer
    int* const ptr = &a;

    // Trying to reassign it to b
    ptr = &b;

    return 0;
}

Output
solution.c: In function ‘main’:
solution.c:11:9: error: assignment of read-only variable ‘ptr’
   11 |     ptr = &b;
      |         ^
```

Pointer to Function

Function Pointer in C

- In C, a function pointer is a type of pointer that stores the address of a function, allowing functions to be passed as arguments and invoked dynamically. It is useful in techniques such as callback functions, event-driven programs, and polymorphism (a concept where a function or operator behaves differently based on the context).
- Let's look at an example:

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main() {

    // Declare a function pointer that matches
       // the signature of add() function
    int (*fptr)(int, int);

    // Assign to add()
```

```
    fptr = &add;

    // Call the function via ptr
    printf("%d", fptr(10, 5));
    return 0;
}

Output
15
```

Function Pointer Declaration

- Function pointers are declared according to the signature of the function they will be pointing to. Below is the generic syntax of function pointer declaration:

```
Syntax: return_type (*pointer_name)(parameter_types);
```

- return_type: The type of value that the function returns.
- parameter_types: The types of the parameters the function takes.
- pointer_name: The name of the function pointer.

```
int (*fptr)(int, int);   /* pointer to a function that takes two ints and returns
int */
```

- The ( ) around *pointer_name is mandatory — without it, compiler treats it as a normal function declaration returning int*, not a pointer.

Initialization

```
fptr = &add;   /* using address operator */
fptr = add;    /* function name alone also works – both are same */
```

- The function signature must match exactly — return type and parameter types must be same as declaration, otherwise compiler throws type mismatch error.

Key Properties

- Points to code (function in code segment), not data.
- Must match exact signature — return type + parameter types.
- Can point to any function with matching signature — swappable at runtime.
- Cannot do arithmetic — no fptr++ or fptr--.
- Can be stored in arrays → useful for function tables (e.g. state machines, command dispatch).

Applications with Examples

Function Pointer as Arguments (Callbacks)

- One of the most useful applications of function pointers is passing functions as arguments to other functions. This allows you to specify which function to call at runtime.

```
#include <stdio.h>

// A simple addition function
int add(int a, int b) {
    return a + b;
}

// A simple subtraction function
int subtract(int a, int b) {
    return a - b;
}

void calc(int a, int b, int (*op)(int, int)) {
    printf("%d\n", op(a, b));
}

int main() {

    // Passing different
    // functions to 'calc'
    calc(10, 5, add);
       calc(10, 5, subtract);
    return 0;
}

Output
15
5
```

Explanation:

The **calc** function accepts a function pointer operation that is used to perform a specific operation (like addition or subtraction) on the two integers a and b. By passing the add or subtract function to calc, the correct function is executed dynamically.

Emulate Member Functions in Structure

- We can create a data member inside structure, but we cannot define a function inside it. But we can define function pointers which in turn can be used to call the assigned functions.

```c
#include <stdio.h>

// Define the Rectangle struct that contains pointers
// to functions as member functions
typedef struct Rect {
    int w, h;
    void (*set)(struct Rect*, int, int);
    int (*area)(struct Rect*);
    void (*show)(struct Rect*);
} Rect;

// Function to find the area
int area(Rect* r) {
    return r->w * r->h;
}

// Function to print the dimensions
void show(Rect* r) {
    printf("Rectangle's Width: %d, "
           "Height: %d\n", r->w, r->h);
}

// Function to set width
// and height (setter)
void set(Rect* r, int w, int h) {
    r->w = w;
    r->h = h;
}

// Initializer/constructor
// for Rectangle
void constructRect(Rect* r) {
    r->w = 0;
    r->h = 0;
    r->set = set;
    r->area = area;
    r->show = show;
}

int main() {
    // Create a Rectangle object
    Rect r;
    constructRect(&r);

    // Use r as a Rectangle
    r.set(&r, 10, 5);
    r.show(&r);
    printf("Rectangle Area: %d", r.area(&r));
    return 0;
}
```

Output

```
Rectangle's Width: 10, Height: 5
Rectangle Area: 50
```

Array of Function Pointers

You can also use function pointers in arrays to implement a set of functions dynamically.

```c
#include <stdio.h>

// Function declarations
int add(int a, int b) {
    return a + b;
}
int sub(int a, int b) {
    return a - b;
}
int mul(int a, int b) {
    return a * b;
}
int divd(int a, int b) {
    if(b!=0)
    return a / b;
    else
    return -1;
}

int main() {

    // Declare an array of function pointers
    int (*farr[])(int, int) = {add, sub, mul, divd};
    int x = 10, y = 5;

    // Dynamically call functions using the array
    printf("Sum: %d\n", farr[0](x, y));
    printf("Difference: %d\n", farr[1](x, y));
    printf("Product: %d\n", farr[2](x, y));
    printf("Divide: %d", farr[3](x, y));

    return 0;
}


Output
Sum: 15
Difference: 5
Product: 50
Divide: 2
```

Multilevel Pointers or Chain of Pointers in C with Examples

- In C, we can create **multi-level pointers** with any number of levels such as – ***ptr3, ****ptr4, ******ptr5 and so on. Most popular of them is **double pointer** (pointer to pointer). It stores the memory address of another pointer. Instead of pointing to a data value, they point to another pointer.

```
#include <stdio.h>

int main() {
    int var = 10;

    // Pointer to int
    int *ptr1 = &var;

    // Pointer to pointer (double pointer)
    int **ptr2 = &ptr1;

    // Accessing values using all three
    printf("var: %d\n", var);
    printf("*ptr1: %d\n", *ptr1);
    printf("**ptr2: %d", **ptr2);

    return 0;
}

Output
var: 10
*ptr1: 10
**ptr2: 10
```

Chain of Pointers in C with Examples

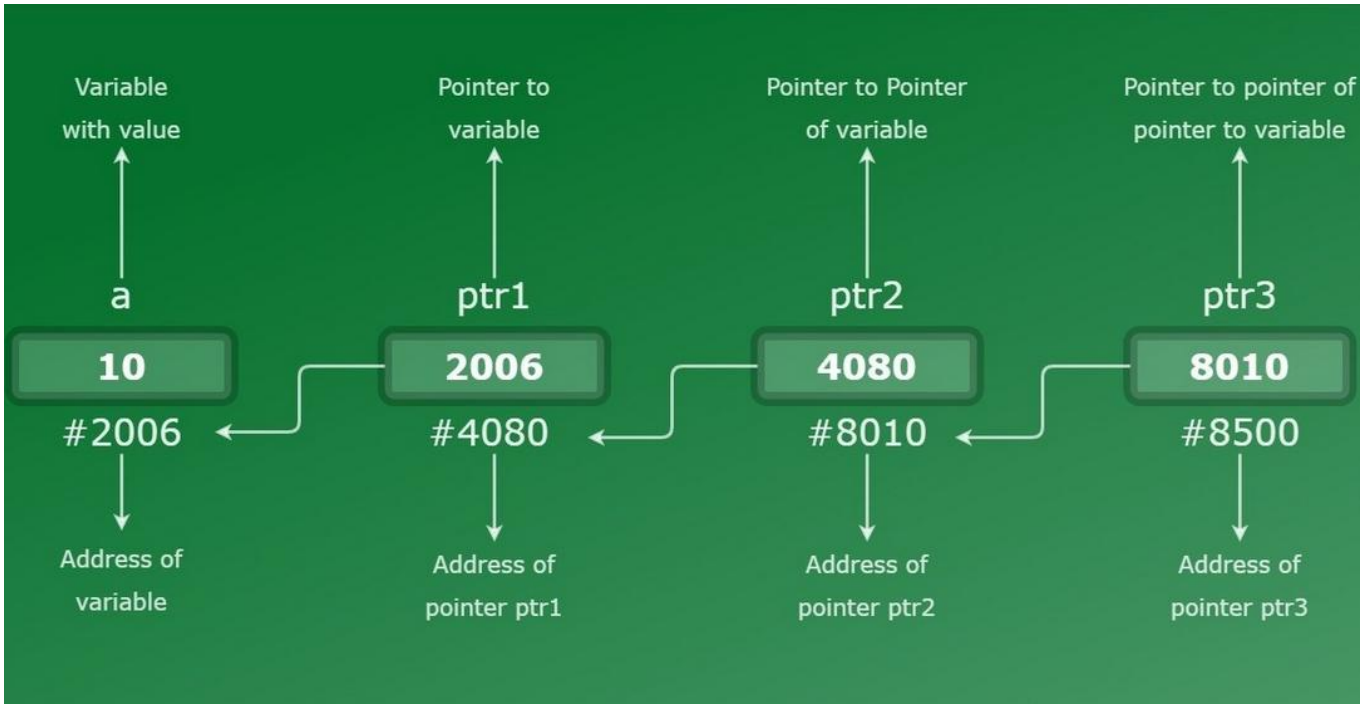
Syntax

```
// level-1 pointer declaration
datatype *pointer;

// level-2 pointer declaration
datatype **pointer;

// level-3 pointer declaration
datatype ***pointer;
.
.
and so on
```

Initialization

```
// initializing level-1 pointer
// with address of variable 'var'
pointer_1 = &var;

// initializing level-2 pointer
// with address of level-1 pointer
pointer_2 = &pointer_1;

// initializing level-3 pointer
// with address of level-2 pointer
pointer_3 = &pointer_2;
.
.
and so on
```

Example 1:



As shown in the diagram, variable 'a' is a normal integer variable which stores integer value 10 and is at location 2006. 'ptr1' is a pointer variable which points to integer variable 'a' and stores its location i.e. 2006, as its value. Similarly ptr2 points to pointer variable ptr1 and ptr3 points at pointer variable ptr2. As every pointer is directly or indirectly pointing to the variable 'a', they all have the same integer value as variable 'a', i.e. 10

Let's understand better with below given code:

```c
#include <stdio.h>
// C program for chain of pointer

int main()
{
    int var = 10;

    // Pointer level-1
    // Declaring pointer to variable var
    int* ptr1;

    // Pointer level-2
    // Declaring pointer to pointer variable *ptr1
    int** ptr2;

    // Pointer level-3
    // Declaring pointer to double pointer **ptr2
    int*** ptr3;

    // Storing address of variable var
    // to pointer variable ptr1
    ptr1 = &var;

    // Storing address of pointer variable
    // ptr1 to level -2 pointer ptr2
    ptr2 = &ptr1;

    // Storing address of level-2 pointer
    // ptr2 to level-3 pointer ptr3
    ptr3 = &ptr2;

    // Displaying values
    printf("Value of variable "
           "var = %d\n",
           var);
    printf("Value of variable var using"
           " pointer ptr1 = %d\n",
           *ptr1);
    printf("Value of variable var using"
           " pointer ptr2 = %d\n",
           **ptr2);
    printf("Value of variable var using"
           " pointer ptr3 = %d\n",
           ***ptr3);

    return 0;
```

```
}

Output

Value of variable var = 10
Value of variable var using pointer ptr1 = 10
Value of variable var using pointer ptr2 = 10
Value of variable var using pointer ptr3 = 10
```

Example 2:

Consider below-given code where we have taken float data type of the variable, so now we have to take same data type for the chain of pointers too. As the pointer and the variable, it is pointing to should have the same data type.

```
#include <stdio.h>
int main()
{
    float var = 23.564327;

    // Declaring pointer variables upto level_4
    float *ptr1, **ptr2, ***ptr3, ****ptr4;

    // Initializing pointer variables
    ptr1 = &var;
    ptr2 = &ptr1;
    ptr3 = &ptr2;
    ptr4 = &ptr3;

    // Printing values
    printf("Value of var = %f\n", var);
    printf("Value of var using level-1"
           " pointer = %f\n",
           *ptr1);
    printf("Value of var using level-2"
           " pointer = %f\n",
           **ptr2);
    printf("Value of var using level-3"
           " pointer = %f\n",
           ***ptr3);
    printf("Value of var using level-4"
           " pointer = %f\n",
           ****ptr4);

    return 0;
}
Output:
```

```
Value of var = 23.564327
Value of var using level-1 pointer = 23.564327
Value of var using level-2 pointer = 23.564327
Value of var using level-3 pointer = 23.564327
Value of var using level-4 pointer = 23.564327
```

Example 3: Updating variable using chained pointer

As we already know that a pointer points to address location of a variable so when we access the value of pointer it'll point to the variable's value. Now to update the value of variable, we can use any level of pointer as ultimately every pointer is directly or indirectly pointing to that variable only. It'll directly change the value present at the address location of variable.

```c
#include <stdio.h>

int main()
{
    // Initializing integer variable
    int var = 10;

    // Declaring pointer variables upto level-3
    int *ptr1, **ptr2, ***ptr3;

    // Initializing pointer variables
    ptr1 = &var;
    ptr2 = &ptr1;
    ptr3 = &ptr2;

    // Printing values BEFORE updation
    printf("Before:\n");
    printf("Value of var = %d\n", var);
    printf("Value of var using level-1"
           " pointer = %d\n",
           *ptr1);
    printf("Value of var using level-2"
           " pointer = %d\n",
           **ptr2);
    printf("Value of var using level-3"
           " pointer = %d\n",
           ***ptr3);

    // Updating var's value using level-3 pointer
    ***ptr3 = 35;

    // Printing values AFTER updation
    printf("After:\n");
```

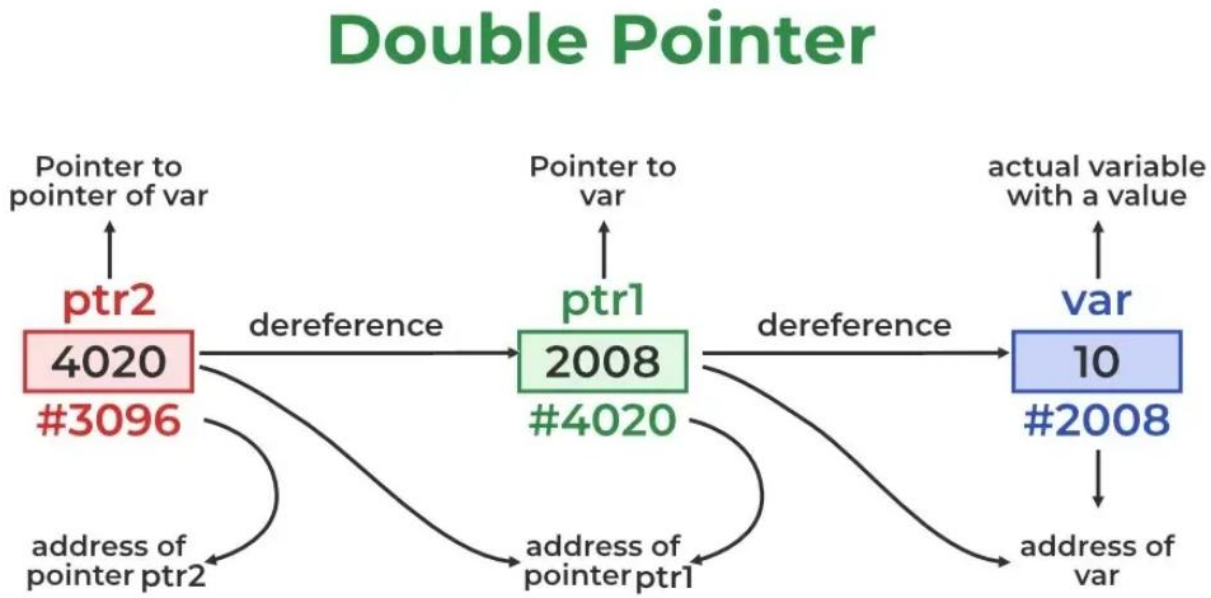
```
    printf("Value of var = %d\n", var);
    printf("Value of var using level-1"
           " pointer = %d\n",
           *ptr1);
    printf("Value of var using level-2"
           " pointer = %d\n",
           **ptr2);
    printf("Value of var using level-3"
           " pointer = %d\n",
           ***ptr3);

    return 0;
}

Output:

Before:
Value of var = 10
Value of var using level-1 pointer = 10
Value of var using level-2 pointer = 10
Value of var using level-3 pointer = 10
After:
Value of var = 35
Value of var using level-1 pointer = 35
Value of var using level-2 pointer = 35
Value of var using level-3 pointer = 35
```

C - Pointer to Pointer (Double Pointer)

- In C, double pointers are those pointers which stores the address of another pointer. The first pointer is used to store the address of the variable, and the second pointer is used to store the address of the first pointer. That is why they are also known as a pointer to pointer.

```
#include <stdio.h>

int main()
{

    // A variable
    int var = 10;

    // Pointer to int
    int *ptr1 = &var;

    // Pointer to pointer (double pointer)
    int **ptr2 = &ptr1;

    printf("var: %d\n", var);
    printf("*ptr1: %d\n", *ptr1);
```

```
    printf("**ptr2: %d", **ptr2);

    return 0;
}
Output
var: 10
*ptr1: 10
**ptr2: 10
```



Size of Double Pointer

- Size of a double pointer is same as size of any other pointer as it also stored address.

```
#include <stdio.h>

int main()
{

    // Defining single and double pointers
    int a = 5;
    int *ptr = &a;
    int **d_ptr = &ptr;

    // Size of double pointer
    printf("%d bytes\n", sizeof(d_ptr));
```

```
    // Size of a single pointer
    printf("%d bytes", sizeof(ptr));

    return 0;
}
Output
8 bytes
8 bytes
```

Create a Dynamic 2D Array

- Double pointers are useful in creating a dynamically sized 2D array where every row can have different number of elements.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int m = 2, n = 3;

    // Create a double pointer
    int **arr;

    // Allocate memory for rows
    arr = (int **)malloc(m * sizeof(int *));

    // Allocate memory for each row
    for (int i = 0; i < m; i++)
        arr[i] = (int *)malloc(n * sizeof(int));

    // Initialize with some values
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            arr[i][j] = i * n + j + 1;
        }
    }

    // Print the array
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
```

```
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }

    // Free allocated memory
    for (int i = 0; i < m; i++)
    {
        free(arr[i]);
    }
    free(arr);

    return 0;
}
```

Output

```
1 2 3
4 5 6
```

Explanation:

A 2D array is an array where each element is essentially a 1D array. This can be implemented using double pointers. The double pointer points to the first element of the 2D array, and each pointer it references points to a dynamically allocated 1D array using malloc().

Pass Array of Strings to Function

```
#include <stdio.h>

// Function that takes array of strings as argument
void print(char **arr, int n)
{
    for (int i = 0; i < n; i++)
        printf("%s\n", *(arr + i));
}

int main()
{
    char *arr[10] = {"Geek", "Geeks", "Geekfor"};
    print(arr, 3);
    return 0;
}
```

Output

```
Geek
Geeks
Geekfor
```

Application of Double Pointers in C

Following are the main uses of pointer to pointers in C:

- They are used in the dynamic memory allocation of multidimensional arrays.
- They can be used to store multilevel data such as the text document paragraph, sentences, and word semantics.
- They are used in data structures to directly manipulate the address of the nodes without copying.
- They can be used as function arguments to manipulate the address stored in the local pointer.

C STRUCTURES

In C, a structure is a user-defined data type that can be used to group items of possibly different types into a single type.

- The struct keyword is used to define a structure. The items in the structure are called its members and they can be of any valid data type.
- Applications of structures involve creating data structures Linked List and Tree. Structures are also used to represent real world objects in a software like Students and Faculty in a college management software.

```
#include <stdio.h>

// Defining a structure
struct A {
    int x;
};

int main() {

    // Creating a structure variable
    struct A a;

       // Initializing member
       a.x = 11;

    printf("%d", a.x);
    return 0;
}

Output
11
```

Basic Operations of Structure

1. Access Structure Members

- To access or modify members of a structure, we use the (.) dot operator. This is applicable when we are using structure variables directly.
- In the case where we have a pointer to the structure, we can also use the arrow operator to access the members.

2. Initialize Structure Members

- Structure members cannot be initialized inside the structure definition. However, they can be initialized at the time of declaring a structure variable using initializer lists.

```c
#include <stdio.h>

// Defining a structure to represent a student
struct Student
{
    char name[50];
    int age;
    float grade;
};

int main()
{

 // Declaring and initializing a structure variable
    struct Student s1 = {"Rahul", 20, 18.5};

 // Designated Initializing another structure
    struct Student s2 = {.age = 18, .name = "Vikas", .grade = 22};

 // Accessing structure members
    printf("%s\t%d\t%.2f\n", s1.name, s1.age, s1.grade);
    printf("%s\t%d\t%.2f\n", s2.name, s2.age, s2.grade);

    return 0;
}

Output
Rahul  20    18.50
Vikas  18    22.00
```

Default Initialization

- Structure members are **not** auto-initialized — they hold garbage values if not set.
- Exception: if you initialize even one member explicitly, all remaining unset members are automatically zero-initialized.

```c
struct Point p = {10};   /* x = 10, y = 0 automatically */
```

Initializer List

Values assigned in order of declaration — position matters.

```c
struct Point p = {10, 20};   /* x = 10, y = 20 in declared order */
```

Designated Initializer List *(C99 and above)*

Members initialized by name — order doesn't matter.

```
struct Point p = { .y = 20, .x = 10 };   /* any order, clear and explicit */
```

- Supported in C (C99+)
- Not supported in C++

Arrays or strings inside a struct **cannot** be assigned using = after declaration — must be initialized at declaration time or use strcpy() for strings, memcpy() for arrays.

3. Copy Structure

Copying structure is simple as copying any other variables. For example, s1 is copied into s2 using assignment operator.

s2 = s1;

But this method only creates a shallow copy of s1 i.e. if the structure **s1** have some dynamic resources allocated by malloc, and it contains pointer to that resource, then only the pointer will be copied to **s2.** If the dynamic resource is also needed, then it has to be copied manually (deep copy).

```
#include <stdio.h>
#include <stdlib.h>

struct Student {
    int id;
      char grade;
};

int main() {
    struct Student s1 = {1, 'A'};

      // Create a copy of student s1
      struct Student s1c = s1;

    printf("Student 1 ID: %d\n", s1c.id);
    printf("Student 1 Grade: %c", s1c.grade);
      return 0;
}
Output
Student 1 ID: 1
Student 1 Grade: A
```

4. Passing Structure to Functions

Structure can be passed to a function in the same way as normal variables. Though, it is recommended to pass it as a pointer to avoid copying a large amount of data.

```c
#include <stdio.h>

// Structure definition
struct A {
    int x;
};

// Function to increment values
void increment(struct A a, struct A* b) {
    a.x++;
      b->x++;
}

int main() {
    struct A a  = { 10 };
      struct A b  = { 10 };

      // Passing a by value and b by pointer
      increment(a, &b);

      printf("a.x: %d \tb.x: %d", a.x, b.x);
    return 0;
}
```

Output

```
a.x: 10      b.x: 11
```

5. typedef for Structures

- The typedef keyword is used to define an alias for the already existing datatype. In structures, we must use the struct keyword along with the structure name to define the variables.
- Sometimes, this increases the length and complexity of the code. We can use the typedef to define some new shorter name for the structure.

```c
#include <stdio.h>

// Defining structure
typedef struct {
    int a;
} str1;

// Another way of using typedef with structures
typedef struct {
    int x;
} str2;

int main() {
```

```
    // Creating structure variables using new names
    str1 var1 = { 20 };
    str2 var2 = { 314 };

    printf("var1.a = %d\n", var1.a);
    printf("var2.x = %d\n", var2.x);
    return 0;
}
Output
var1.a = 20
var2.x = 314
```

Explanation:

In this code, **str1** and **str2** are defined as aliases for the unnamed structures, allowing the creation of structure variables (**var1** and **var2**) using these new names. This simplifies the syntax when declaring variables of the structure.

Size of Structures

- Size of a struct is not always the sum of its members — because of padding and packing.

Padding

- Compiler inserts extra empty bytes between members to align data on natural boundaries — so CPU can read it in fewer cycles.

```
struct A {
    char  a;    /* 1 byte  + 3 bytes padding inserted */
    int   b;    /* 4 bytes */
};
/* sizeof(A) = 8, not 5 */
```

Packing

- Forces compiler to remove padding gaps — members stored back to back. Saves memory but may slow down CPU access.

```
/* Method 1 */
#pragma pack(1)
struct A { char a; int b; };   /* sizeof = 5, no padding */

/* Method 2 */
struct A { char a; int b; } __attribute__((packed));   /* sizeof = 5 */
```

	Padding	Packing
Extra bytes	✅ Added	❌ Removed
CPU speed	Faster	Slightly slower
Memory	More used	Less used
Use case	General use	Embedded / protocol buffers

Nested Structures

- A structure containing another structure as its member. Used to group related data together logically.

Two ways to nest:

1. Embedded — inner struct declared inside the outer struct

```
struct parent {
    int a;
    struct child {   /* child declared inside parent itself */
        int x;
        char c;
    } b;             /* b is the member of type child */
};
```

2. Separate — both structs declared independently

```
/* child declared separately */
struct child {
    int x;
    char c;
};

/* child used as a member inside parent */
struct parent {
    int a;
    struct child b;   /* b is of type child */
};
```

Separate nesting is preferred — cleaner and reusable across multiple structs.

Accessing Nested Members

Use . dot operator twice — once for outer struct, once for inner struct.

```
#include <stdio.h>

struct child {
    int x;
    char c;
};

struct parent {
    int a;
    struct child b;   /* child struct nested as member b */
};

int main() {

    /* initializing parent:
       25 goes to p.a
       195 goes to p.b.x
       'A' goes to p.b.c  */
    struct parent p = { 25, {195, 'A'} };

    printf("p.a   = %d\n", p.a);     /* accesses parent member directly */
    printf("p.b.x = %d\n", p.b.x);  /* accesses child member x via
parent.child.member */
    printf("p.b.c = %c\n", p.b.c);  /* accesses child member c via
parent.child.member */

    return 0;
}
Output
p.a   = 25
p.b.x = 195
p.b.c = A
```

Note : Initialization { 25, 195, 'A' } is technically valid in some compilers but not standard. Correct way is { 25, {195, 'A'} } — inner struct values wrapped in their own { }.

Structure Pointer

- A pointer to a structure allows us to access structure members using the (->) arrow operator instead of the dot operator.

```
#include <stdio.h>

// Structure declaration
struct Point {
    int x, y;
};

int main() {
    struct Point p = { 1, 2 };

    // ptr is a pointer to structure p
    struct Point* ptr = &p;

    // Accessing structure members using structure pointer
    printf("%d %d", ptr->x, ptr->y);

    return 0;
}

Output
1 2
```

Explanation:

In this example, ptr is a pointer to the structure Point. It holds the address of the structure variable p. The structure members x and y are accessed using the -> operator, which is used to dereference the pointer and access the members of the structure.

Self-Referential Structures in C

A structure that contains a pointer to another object of the same structure type as one of its members. This allows multiple structure objects to be linked together.

1. Contains one or more pointers to the same structure type.
2. Foundation of dynamic data structures like linked lists, trees, graphs.

```
struct structure_name {
    data_type member1;
    data_type member2;
    struct structure_name *pointer_name;  /* pointer to same struct type */
};
```

```c
#include <stdio.h>

struct Node {
    int data1;
    char data2;
    struct Node *link;   /* self-referential pointer – points to another Node */
};

int main() {
    struct Node obj;
    obj.data1 = 10;
    obj.data2 = 'A';
    obj.link  = NULL;    /* always initialize to NULL before use */
    return 0;
}
```

Link is a pointer to another object of type Node — making Node a self-referential structure.

⚠️ Always initialize self-referential pointers to NULL before using them — uninitialized pointers cause undefined behavior.

Types of Self-Referential Structures

1. Single Link — One self-pointer

- Each node connects to only one other node. Used in singly linked lists.

```c
#include <stdio.h>


struct node {

    int data1;

    char data2;

    struct node *link;   /* points to the next node */

};


int main() {


    struct node ob1;         /* Node 1 */

    ob1.link  = NULL;        /* initialize pointer to NULL */

    ob1.data1 = 10;

    ob1.data2 = 'A';         /* corrected: data2 is char, should be char value */
```

```
    struct node ob2;         /* Node 2 */
    ob2.link  = NULL;        /* last node points to NULL – end of chain */
    ob2.data1 = 30;
    ob2.data2 = 'B';

    ob1.link = &ob2;         /* link ob1 to ob2 – ob1 now points to ob2 */

    /* accessing ob2's members through ob1's link pointer
       -> operator dereferences pointer and accesses member */
    printf("%d\n",  ob1.link->data1);   /* prints 30 */
    printf("%c\n",  ob1.link->data2);   /* prints B  */

    return 0;
}

Output
30
B
```

- ob1.link stores address of ob2.
- ob1.link->data1 dereferences the pointer and accesses ob2's member.
- Last node's link = NULL marks end of the chain.

2. Multiple Links — Two or more self-pointers

Each node connects to multiple nodes — forward and backward. Used in doubly linked lists, trees, graphs.

```
#include <stdio.h>

struct node {
    int data;
    struct node *prev_link;   /* points to previous node */
    struct node *next_link;   /* points to next node */
};

int main() {

    struct node ob1;             /* Node 1 */
```

```
    ob1.prev_link = NULL;        /* no previous node for first node */
    ob1.next_link = NULL;
    ob1.data = 10;

    struct node ob2;             /* Node 2 */
    ob2.prev_link = NULL;
    ob2.next_link = NULL;
    ob2.data = 20;

    struct node ob3;             /* Node 3 */
    ob3.prev_link = NULL;
    ob3.next_link = NULL;        /* no next node for last node */
    ob3.data = 30;

    /* setting forward links: ob1 -> ob2 -> ob3 */
    ob1.next_link = &ob2;
    ob2.next_link = &ob3;

    /* setting backward links: ob3 -> ob2 -> ob1 */
    ob2.prev_link = &ob1;
    ob3.prev_link = &ob2;

    /* forward traversal starting from ob1 */
    printf("%d\t", ob1.data);                          /* ob1 directly */
    printf("%d\t", ob1.next_link->data);               /* ob1 -> ob2   */
    printf("%d\n", ob1.next_link->next_link->data);    /* ob1 -> ob3   */

    /* traversal starting from ob2 in both directions */
    printf("%d\t", ob2.prev_link->data);               /* ob2 -> ob1   */
    printf("%d\t", ob2.data);                          /* ob2 directly */
    printf("%d\n", ob2.next_link->data);               /* ob2 -> ob3   */

    /* backward traversal starting from ob3 */
    printf("%d\t", ob3.prev_link->prev_link->data);    /* ob3 -> ob1   */
    printf("%d\t", ob3.prev_link->data);               /* ob3 -> ob2   */
    printf("%d\n", ob3.data);                          /* ob3 directly */

    return 0;
}
Output
10    20    30
10    20    30
10    20    30
```

- Each node has prev_link and next_link — allows traversal in both directions.
- First node's prev_link = NULL, last node's next_link = NULL.
- Chaining -> twice (ob1.next_link->next_link->data) jumps two nodes forward

Applications

Data Structure	How self-referential struct is used
Singly linked list	Each node has one `next` pointer
Doubly linked list	Each node has `next` and `prev` pointers
Stack	Nodes linked via `next` , top tracked separately
Queue	Nodes linked via `next` , front and rear tracked
Binary tree	Each node has `left` and `right` child pointers
Graph	Each node has pointer to list of connected nodes

BIT FIELDS IN C

- In C, we can specify the size (in bits) of the structure and union members. The idea of bit-field is to use memory efficiently when we know that the value of a field or group of fields will never exceed a limit or is within a small range. C Bit fields are used when the storage of our program is limited.

Need of Bit Fields in C

- Reduces memory consumption.
- To make our program more efficient and flexible.
- Easy to Implement.

Declaration of C Bit Fields

Bit-fields are variables that are defined using a predefined width or size. Format and the declaration of the bit-fields in C are shown below:

Syntax of C Bit Fields

```
struct
{
    data_type member_name : width_of_bit-field;
};
```

- **data_type:** It is an integer type that determines the bit-field value which is to be interpreted. The type may be int, signed int, or unsigned int.
- **member_name:** The member name is the name of the bit field.
- **width_of_bit-field:** The number of bits in the bit-field. The width must be less than or equal to the bit width of the specified type.

Applications of C Bit Fields

- If storage is limited, we can go for bit-field.
- When devices transmit status or information encoded into multiple bits for this type of situation bit-field is most efficient.
- Encryption routines need to access the bits within a byte in that situation bit-field is quite useful.

Example of C Bit Fields

- In this example, we compare the size difference between the structure that does not specify bit fields and the structure that has specified bit fields.

Structure Without Bit Fields

Consider the following declaration of date without the use of bit fields.

```
// C Program to illustrate the structure without bit field
#include <stdio.h>

// A simple representation of the date
struct date {
    unsigned int d;
    unsigned int m;
    unsigned int y;
};

int main()
{
    // printing size of structure
    printf("Size of date is %lu bytes\n",
           sizeof(struct date));
    struct date dt = { 31, 12, 2014 };
    printf("Date is %d/%d/%d", dt.d, dt.m, dt.y);
}
Output
Size of date is 12 bytes
Date is 31/12/2014
```

The above representation of 'date' takes 12 bytes on a compiler whereas an unsigned int takes 4 bytes. Since we know that the value of d is always from 1 to 31, and the value of m is from 1 to 12, we can optimize the space using bit fields.

Structure with Bit Field

The below code defines a structure named date with a single member month. The month member is declared as a bit field with 4 bits.

However, if the same code is written using signed int and the value of the fields goes beyond the bits allocated to the variable, something interesting can happen.

Below is the same code but with signed integers:

```
// C program to demonstrate use of Bit-fields
#include <stdio.h>

// Space optimized representation of the date
struct date {
    // d has value between 0 and 31, so 5 bits
    // are sufficient
    int d : 5;

    // m has value between 0 and 15, so 4 bits
    // are sufficient
    int m : 4;

    int y;
};

int main()
{
    printf("Size of date is %lu bytes\n",
           sizeof(struct date));
    struct date dt = { 31, 12, 2014 };
    printf("Date is %d/%d/%d", dt.d, dt.m, dt.y);
    return 0;
}
Output
Size of date is 8 bytes
Date is -1/-4/2014
```

Explanation

The output comes out to be negative. What happened behind is that the value 31 was stored in 5 bit signed integer which is equal to 11111. The MSB is a 1, so it's a negative number and you need to calculate the 2's complement of the binary number to get its actual value which is what is done internally.

By calculating 2's complement you will arrive at the value 00001 which is equivalent to the decimal number 1 and since it was a negative number you get a -1. A similar thing happens to 12 in which case you get a 4-bit representation as 1100 and on calculating 2's complement you get the value of -4.

Interesting Facts About C Bit Fields

1. A special unnamed bit field of size 0 is used to force alignment on the next boundary.

Example: The below code demonstrates how to force alignment to the next memory boundary using bit fields.

```c
// C Program to illustrate the use of forced alignment in
// structure using bit fields
#include <stdio.h>

// A structure without forced alignment
struct test1 {
    unsigned int x : 5;
    unsigned int y : 8;
};

// A structure with forced alignment
struct test2 {
    unsigned int x : 5;
    unsigned int : 0;
    unsigned int y : 8;
};

int main()
{
    printf("Size of test1 is %lu bytes\n",
           sizeof(struct test1));
    printf("Size of test2 is %lu bytes\n",
           sizeof(struct test2));
    return 0;
}
Output
Size of test1 is 4 bytes
Size of test2 is 8 bytes
```

2. We cannot have pointers to bit field members as they may not start at a byte boundary.

Example: The below code demonstrates that taking the address of a bit field member directly is not allowed.

```c
// C program to demonstrate that the pointers cannot point
// to bit field members
#include <stdio.h>
struct test {
    unsigned int x : 5;
    unsigned int y : 5;
    unsigned int z;
};
int main()
{
    struct test t;
```

```
    // Uncommenting the following line will make
    // the program compile and run
    printf("Address of t.x is %p", &t.x);

    // The below line works fine as z is not a
    // bit field member
    printf("Address of t.z is %p", &t.z);
    return 0;
}
Output
prog.c: In function 'main':
prog.c:14:1: error: cannot take address of bit-field 'x'
 printf("Address of t.x is %p", &t.x);
 ^
```

3. It is implementation-defined to assign an out-of-range value to a bit field member.

Example: The below code demonstrates the usage of bit fields within a structure and assigns an out-of-range value to one of the bit field members.

```
// C Program to show what happends when out of range value
// is assigned to bit field member
#include <stdio.h>

struct test {
    // Bit-field member x with 2 bits
    unsigned int x : 2;
    // Bit-field member y with 2 bits
    unsigned int y : 2;
    // Bit-field member z with 2 bits
    unsigned int z : 2;
};

int main()
{
    // Declare a variable t of type struct test
    struct test t;
    // Assign the value 5 to x (2 bits)
    t.x = 5;

    // Print the value of x
    printf("%d", t.x);

    return 0;
}
Output
Implementation-Dependent
```

4. Array of bit fields is not allowed.

Example: The below C program defines an array of bit fields and fails in the compilation.

```
// C Program to illustrate that we cannot have array bit
// field members
#include <stdio.h>


// structure with array bit field
struct test {
    unsigned int x[10] : 5;
};


int main() {}
```

Output

```
prog.c:3:1: error: bit-field 'x' has invalid type
 unsigned int x[10]: 5;
 ^
```

UNIONS IN C

- A union is a user-defined data type that can hold different data types, similar to a structure.
- Unlike structures, all members of a union are stored in the same memory location. Since all members share the same memory, changing the value of one member overwrites the value of the others.
- Union members can be accessed using the dot (.) operator. Example: u.member.
- Unions are useful when you want to save memory by storing different types of data in the same memory space.

```
#include <stdio.h>

// Define a union with
// different data types
union Student {
    int rollNo;
    float height;
    char firstLetter;
};

int main() {

    // Declare a union variable
    union Student data;

    // Assign and print the roll number
    data.rollNo = 21;
    printf("%d\n", data.rollNo);
    data.height = 5.2;
    printf("%.2f\n", data.height);
    data.firstLetter = 'N';
    printf("%c", data.firstLetter);

    return 0;
}

Output
21
5.20
N
```

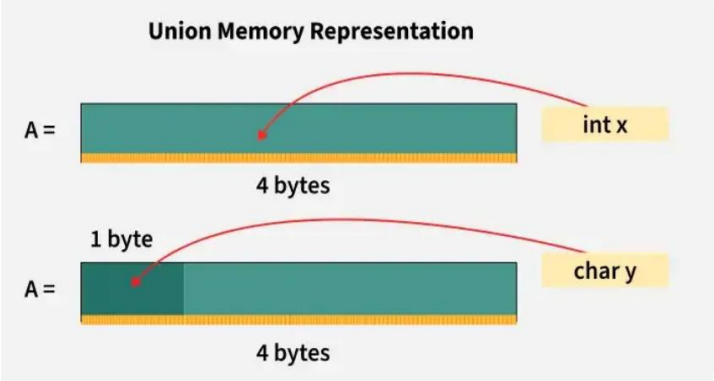
Union Declaration

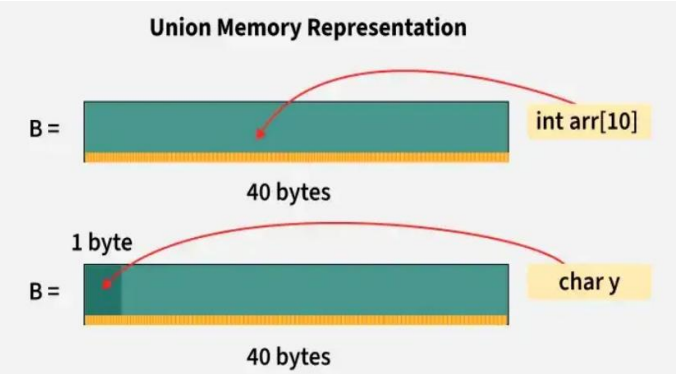
- A union is declared similarly to a structure. Provide the name of the union and define its member variables.

Size of Union

- The size of the union will always be equal to the size of the largest member of the union. All the less-sized elements can store the data in the same space without any overflow.

```c
#include <stdio.h>

union A{
    int x;
    char y;
};

union B{
    int arr[10];
    char y;
};


int main() {

    // Finding size using sizeof() operator
    printf("Sizeof A: %ld\n", sizeof(union A));
    printf("Sizeof B: %ld\n", sizeof(union B));
    return 0;
}

Output
Sizeof A: 4
Sizeof B: 40
```





Nested Union

- In C, we can define a union inside another union like **structure**. This is called **nested union** and is commonly used when you want to efficiently organize and access related data while sharing memory among its members.

```c
#include <stdio.h>


// Define a union with
```

```c
// different data types
union Student {

    int rollNo;

    union Academic{

        int marks;

    } performance;

};


int main() {


    // Declare a union variable
    union Student abc;


    // Assign and print the
    // roll number
    abc.rollNo = 21;

    printf("%d\n", abc.rollNo);


    // Assign and print the
    // member of inner union
    abc.performance.marks = 91;

    printf("%d", abc.performance.marks);

    return 0;

}
```

Output

```
21

91
```

Anonymous Union

- An anonymous union in C is a union that does not have a name. Instead of accessing its members through a named union variable, you can directly access the members of the anonymous union. This is useful when you want to access the union members directly within a specific scope, without needing to declare a union variable.

```c
#include <stdio.h>

// Define a union with
// different data types
struct Student {
    int rollNo;

    // Anonymous union
    union {
        int marks;
    } performance;
};

int main() {

    // Declare a structure variable
    struct Student abc;

    abc.rollNo = 21;
    printf("%d\n", abc.rollNo);

    // Assign and print the member of anonymous union
    abc.performance.marks = 91;
    printf("%d", abc.performance.marks);

    return 0;
}
```

Output

```
21
91
```

Applications of Union in C

- Memory-efficient: Multiple data types share the same memory, reducing overall memory usage.
- Hardware mapping: Useful for registers with different interpretations (e.g., status vs. command).
- Single-type data: Ideal for data that can be one of several types but not simultaneously (e.g., message ID or string).

Similarities Between Structure and Union

Structures and unions are also similar in some aspects listed below:

- Both are user-defined data types used to store data of different types as a single unit.
- Their members can be objects of any type, including other structures and unions or arrays. A member can also consist of a bit field.
- Both structures and unions support only assignment = and sizeof operators. The two structures or unions in the assignment must have the same members and member types.

- A structure or a union can be passed by value to functions and returned by value by functions. The argument must have the same type as the function parameter. A structure or union is passed by value just like a scalar variable as a corresponding parameter.
- '.' operator or selection operator, which has one of the highest precedences, is used for accessing member variables inside both the user-defined datatypes.

Parameter	Structure	Union
Definition	A structure is a user-defined data type that groups different data types into a single entity.	A union is a user-defined data type that allows storing different data types at the same memory location.
Keyword	The keyword **struct** is used to define a structure	The keyword **union** is used to define a union
Size	The size is the sum of the sizes of all members, with padding if necessary.	The size is equal to the size of the largest member, with possible padding.
Memory Allocation	Each member within a structure is allocated unique storage area of location.	Memory allocated is shared by individual members of union.
Data Overlap	No data overlap as members are independent.	Full data overlap as members shares the same memory.
Accessing Members	Individual member can be accessed at a time.	Only one member can be accessed at a time.

ENUMERATION (OR ENUM) IN C

- A user-defined data type that assigns meaningful names to a set of integer constants. Makes code easier to read and maintain.

```
enum enum_name {
    n1, n2, n3, ...
};
```

By default n1 = 0, n2 = 1, each next name increments by 1.

Declaration

```
enum calculate {
    SUM,         /* 0 */
    DIFFERENCE,  /* 1 */
    PRODUCT,     /* 2 */
    QUOTIENT     /* 3 */
};
```

Uppercase names preferred for easy identification.
Enum constants must be **unique within the same scope.**

```
   enum calculate { SUM, DIFFERENCE, PRODUCT, QUOTIENT };
   enum item      { PRODUCT, SERVICE };   /* ERROR: PRODUCT already defined */
```

```
error: redeclaration of enumerator 'PRODUCT'
```

Create Enum Variables

```
#include <stdio.h>

enum direction {
    EAST,    /* 0 */
    NORTH,   /* 1 */
    WEST,    /* 2 */
    SOUTH    /* 3 */
};

int main() {
```

```
    /* creating enum variable and assigning enum constant */
    enum direction dir = NORTH;
    printf("%d\n", dir);   /* prints 1 */

    /* assigning integer directly – valid but not recommended
       defeats the purpose of using meaningful names */
    dir = 3;
    printf("%d\n", dir);   /* prints 3 – same as SOUTH */

    return 0;
}

Output
1
3
```

Assign Values Manually

- Values can be assigned in any order. Unassigned names auto-increment from the previous value.

```
#include <stdio.h>

enum enm {
    a = 3,   /* manually set to 3 */
    b = 2,   /* manually set to 2 */
    c        /* auto assigned 2+1 = 3 */
};

int main() {

    enum enm v1 = a;   /* v1 = 3 */
    enum enm v2 = b;   /* v2 = 2 */
    enum enm v3 = c;   /* v3 = 3 – same value as a, two names can share same value */

    printf("%d %d %d\n", v1, v2, v3);   /* prints 3 2 3 */

    return 0;
}

Output

3 2 3
```

Two enum names **can share the same value** — not an error.

Size of Enum

Enum definition itself occupies no memory. Only enum variables are allocated memory — typically same size as int (4 bytes). Some compilers may use smaller types.

```
#include <stdio.h>

enum direction {
    EAST, NORTH, WEST, SOUTH
};

int main() {

    enum direction dir = NORTH;

    /* sizeof returns size of enum variable in bytes
       %zu is correct format specifier for sizeof output */
    printf("%zu bytes\n", sizeof(dir));   /* prints 4 bytes */

    return 0;
}
Output:
4 bytes
```

Enum with Typedef

- typedef creates an alias for the enum — avoids repeating the enum keyword every time.

```
#include <stdio.h>


/* typedef gives alias 'Dirctn' to 'enum direction'
   now we can use Dirctn directly instead of enum direction */
typedef enum direction {

    EAST,    /* 0 */

    NORTH,   /* 1 */

    WEST,    /* 2 */

    SOUTH    /* 3 */

} Dirctn;


int main() {


    Dirctn dir = NORTH;   /* no need to write 'enum direction' */
```

```
    printf("%d\n", dir);  /* prints 1 */


    return 0;

}
```

Output
1

Enum in switch-case — most common real-world use of enums. Compiler warns if a case is missing for any enum value.

```
#include <stdio.h>

typedef enum { EAST, NORTH, WEST, SOUTH } Direction;

int main() {

    Direction dir = NORTH;

    /* switch with enum — compiler warns if any enum value is unhandled
       makes state machines clean and readable */
    switch (dir) {
        case EAST:  printf("Going East\n");  break;
        case NORTH: printf("Going North\n"); break;
        case WEST:  printf("Going West\n");  break;
        case SOUTH: printf("Going South\n"); break;
    }

    return 0;
}
```

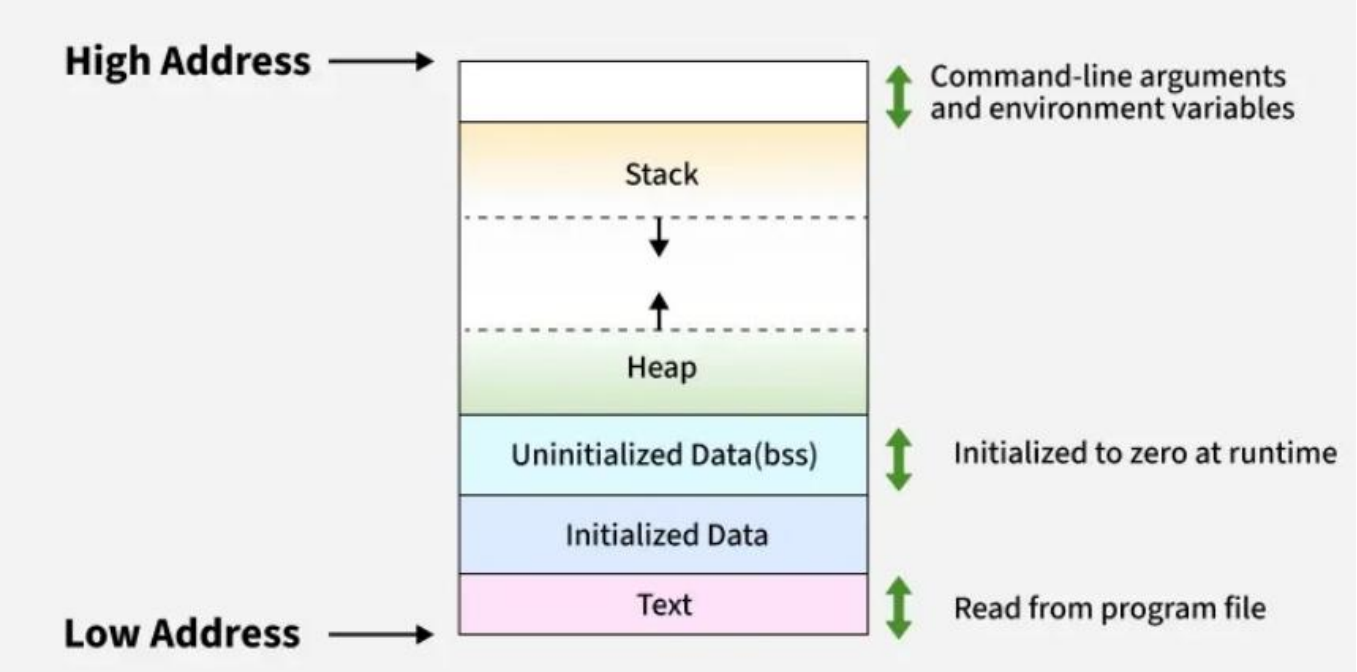
Output
Going North

Applications

Use case	Example
State machines	`STATE_IDLE, STATE_RUNNING, STATE_ERROR`
Error codes	`ERR_NONE, ERR_TIMEOUT, ERR_OVERFLOW`
Menu / user input	`MENU_START, MENU_SETTINGS, MENU_EXIT`
File permissions	`PERM_READ, PERM_WRITE, PERM_EXECUTE`
Switch-case dispatch	Clean handler per enum value

MEMORY LAYOUT OF C PROGRAMS

- The memory layout of a program shows how its data is stored in memory during execution. It helps developers understand and manage memory efficiently.
- Memory is divided into sections such as code, data, heap, and stack.
- Knowing the memory layout is useful for optimizing performance, debugging and prevent errors like segmentation fault and memory leak.



1. Text Segment

- The text segment (or code segment) stores the executable code of the program like program's functions and instructions.
- The segment is usually read-only to prevent accidental modification during execution.
- It is typically stored in the lower part of memory.
- The size of the text segment depends on the number of instructions and the program's complexity.

2. Data Segment

- The data segment stores global and static variables of the program.
- Variables in this segment retain their values throughout program execution.
- The size of the data segment depends on the number and type of global/static variables.
- It is divided into initialized and uninitialized (BSS) sections.

1. Initialized Data Segment

As the name suggests, it is the part of the data segment that contains global and static variables that have been initialized by the programmer.

```c
#include <stdio.h>

// Global variables (stored in initialized data segment)
int globalVar = 10;
char message[] = "Hello";

int main()
{
    // Static variable (also stored in initialized data segment)
    static int staticVar = 20;

    printf("Global variable: %d\n", globalVar);
    printf("Static variable: %d\n", staticVar);
    printf("Message: %s\n", message);

    return 0;
}
Output
Global variable: 10
Static variable: 20
Message: Hello
```

The above variables a and b will be stored in the Initialized Data Segment.

2. Uninitialized Data Segment (BSS)

1. The uninitialized data segment is often called the BSS segment.
2. It stores global and static variables that are not initialized by the programmer.
3. These variables are automatically initialized to zero by the system at runtime.

```c
#include <stdio.h>

// Global uninitialized variables (stored in BSS segment)
int globalVar;
char message[50];

int main()
{
    // Static uninitialized variable (also stored in BSS)
    static int staticVar;

    // Assigning values at runtime
    globalVar = 10;
    staticVar = 20;
    snprintf(message, sizeof(message), "Hello BSS");

    printf("Global variable: %d\n", globalVar);
    printf("Static variable: %d\n", staticVar);
    printf("Message: %s\n", message);
```

```
    return 0;
}

Output
Global variable: 10
Static variable: 20
Message: Hello BSS
```

Heap Segment

- The heap segment is used for dynamic memory allocation.
- It starts at the end of the BSS segment and grows towards higher memory addresses.
- Memory in the heap is managed using functions like malloc(), realloc(), and free().
- The heap is shared by all shared libraries and dynamically loaded modules in a process.

```
#include <stdio.h>
#include <stdlib.h>

int main() {

    // Create an integer pointer
    int *ptr = (int*) malloc(sizeof(int) * 10);
    return 0;
}
```

Stack Segment

- The stack stores local variables, function parameters, and return addresses for each function call.
- Each function call creates a stack frame in this segment.
- The stack is usually at higher memory addresses and grows opposite to the heap.
- When the stack and heap meet, the program's free memory is exhausted.

```
#include <stdio.h>

void func() {

    // Stored in the stack
    int local_var = 10;
}

int main() {
    func();
    return 0;
}
```

Practical Examples

The size(1) command in MinGW reports the sizes (in bytes) of the text, data, and bss segments of a binary file.

1. Check the following simple C program

```
#include <stdio.h>

int main() {
    return 0;
}
```

Memory Layout

```
gcc memory-layout.c -o memory-layout
size memory-layout
text data bss dec hex filename
960 248 8 1216 4c0 memory-layout
```

2. Let us add one global variable in the program, now check the size of bss

```
#include <stdio.h>

// Uninitialized variable stored in bss
int global;
int main() {
    return 0;
}
```

Memory Layout

```
gcc memory-layout.c -o memory-layout
size memory-layout
text data bss dec hex filename
960 248 12 1220 4c4 memory-layout
```

3. Let us add one static variable which is also stored in bss.

```
#include <stdio.h>

// Uninitialized variable stored in bss
int global;
int main() {

      // Uninitialized static variable stored in bss
    static int i;
    return 0;
}
```

Memory Layout

```
gcc memory-layout.c -o memory-layout
size memory-layout
text data bss dec hex filename
960 248 16 1224 4c8 memory-layout
```

4. Let us initialize the static variable which will then be stored in the Data Segment (DS)

```
#include <stdio.h>

// Uninitialized variable stored in bss
int global;
int main(void) {

      // Initialized static variable stored in DS
    static int i = 100;
    return 0;
}
```

Memory Layout

```
gcc memory-layout.c -o memory-layout
size memory-layout
text data bss dec hex filename
960 252 12 1224 4c8 memory-layout
```

5. Let us initialize the global variable which will then be stored in the Data Segment (DS)

```
#include <stdio.h>

// initialized global variable stored in DS
int global = 10;
int main() {

      // Initialized static variable stored in DS
    static int i = 100;
    return 0;
}
```

Memory Layout

```
gcc memory-layout.c -o memory-layout
size memory-layout
text data bss dec hex filename
960 256 8 1224 4c8 memory-layout
```

Example to Verify the Memory Layout

```
#include <stdio.h>
#include <stdlib.h>

// Global variable
int gvar = 66;

// Constant global variable
const int cgvar = 1010;

// uninitialized global variable
int ugvar;

void foo() {

    // Local variable
    int lvar = 1;
    printf("Address of lvar:\t%p", (void*)&lvar);
}

int main() {

    // Heap variable
    int *hvar = (int*)malloc(sizeof(int));
```

```
    // Checking and comparing address of different
    // elements of program that should be stored in
    // different segements of the memory
    printf("Address of foo:\t\t%p\n", (void*)&foo);
    printf("Address of cgvar:\t%p\n", (void*)&cgvar);
    printf("Address of gvar:\t%p\n", (void*)&gvar);
    printf("Address of ugvar:\t%p\n", (void*)&ugvar);
    printf("Address of hvar:\t%p\n", (void*)hvar);
    foo();

       return 0;
}


Output
Address of foo:           0x4011d0
Address of cgvar:   0x402084
Address of gvar:    0x404020
Address of ugvar:   0x404028
Address of hvar:    0x119592a0
Address of lvar:    0x7ffe8289c66c
Comparing above addresses, we can see than it roughly matches the memory layout
discussed above.
```

DYNAMIC MEMORY ALLOCATION IN C

Dynamic memory allocation allows a programmer to allocate, resize, and free memory at runtime. Key advantages include.

- Memory is allocated on the heap area instead of stack.
- Array size can be increased or decreased as needed.
- Memory persists even after the function that allocated it finishes, allowing functions to return pointers to it. This is different from stack allocated variables as it is not safe to return address of those variable.
- The malloc(), calloc(), realloc() and free() functions are the primary tools for dynamic memory management in C, they are part of the C Standard Library and are defined in the <stdlib.h> header file.

malloc()

- The malloc() (stands for memory allocation) function is used to allocate a single block of contiguous memory on the heap at runtime. The memory allocated by malloc() is uninitialized, meaning it contains garbage values.

- Assume that we want to create an array to store 5 integers. Since the size of int is 4 bytes, we need 5 * 4 bytes = 20 bytes of memory. This can be done as shown:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(20);

    // Populate the array
    for (int i = 0; i < 5; i++)
        ptr[i] = i + 1;

    // Print the array
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    return 0;
}
Output
1 2 3 4 5
```

Explanation:

In the above malloc call, we hardcoded the number of bytes we need to store 5 integers. But we know that the size of the integer in C depends on the architecture. So, it is better to use the sizeof operator to find the size of type you want to store.

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr = (int *)malloc(sizeof(int) * 5);

    // Populate the array
    for (int i = 0; i < 5; i++)
        ptr[i] = i + 1;

    // Print the array
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    return 0;
}
```

Output

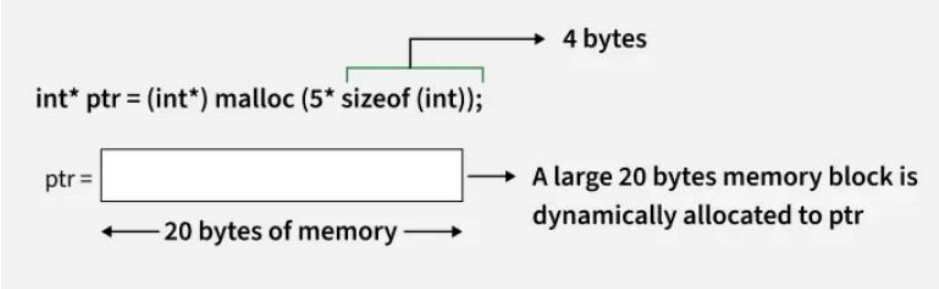
```
1 2 3 4 5
```

Moreover, if there is no memory available, the malloc will fail and return NULL. So, it is recommended to check for failure by comparing the ptr to NULL.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(sizeof(int) * 5);

    // Checking if failed or pass
    if (ptr == NULL) {
        printf("Allocation Failed");
        exit(0);
    }

    // Populate the array
    for (int i = 0; i < 5; i++)
        ptr[i] = i + 1;

    // Print the array
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    return 0;
}
```

Output

```
1 2 3 4 5
```

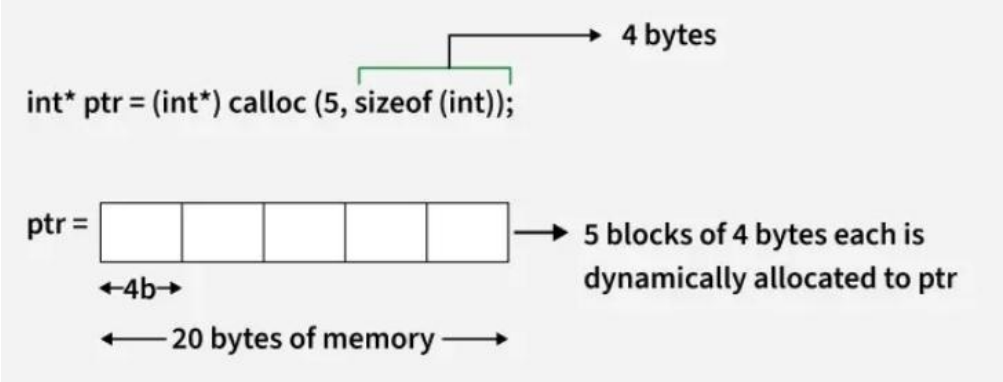


This function returns a void pointer to the allocated memory that needs to be converted to the pointer of required type to be usable. If allocation fails, it returns NULL pointer.

calloc()

- The calloc() (stands for contiguous allocation) function is similar to malloc(), but it initializes the allocated memory to zero. It is used when you need memory with default zero values.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)calloc(5, sizeof(int));

    // Checking if failed or pass
    if (ptr == NULL) {
        printf("Allocation Failed");
        exit(0);
    }

    // No need to populate as already
    // initialized to 0

    // Print the array
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    return 0;
}
Output
0 0 0 0 0
```



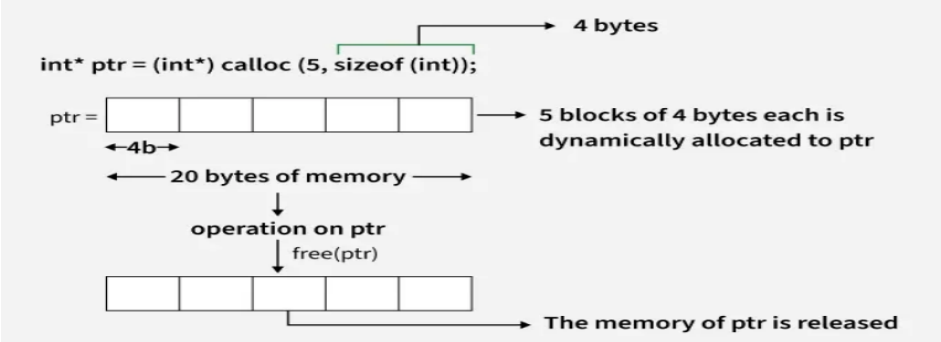
This function also returns a void pointer to the allocated memory that is converted to the pointer of required type to be usable. If allocation fails, it returns NULL pointer.

free()

- The memory allocated using functions malloc() and calloc() is not de-allocated on their own. The free() function is used to release dynamically allocated memory back to the operating system. It is essential to free memory that is no longer needed to avoid memory leaks.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)calloc(5, sizeof(int));

    // Do some operations.....
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);

    // Free the memory after completing
    // operations
    free(ptr);

    return 0;
}
Output
0 0 0 0 0
```

After calling free(), it is a good practice to set the pointer to NULL to avoid using a "dangling pointer," which points to a memory location that has been deallocated.

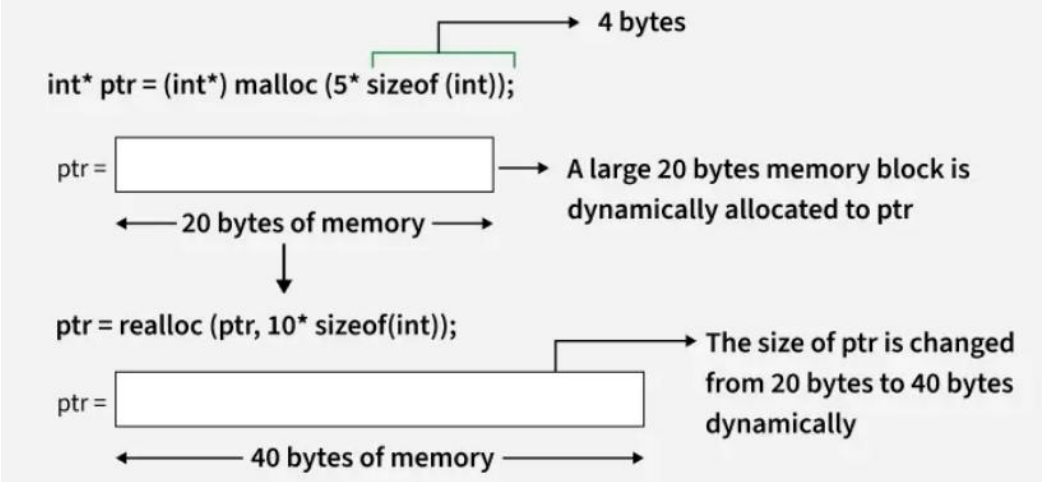


After freeing a memory block, the pointer becomes invalid, and it is no longer pointing to a valid memory location.

realloc()

- realloc() function is used to resize a previously allocated memory block. It allows you to change the size of an existing memory allocation without needing to free the old memory and allocate a new block.
- Suppose we initially allocate memory for 5 integers but later need to expand the array to hold 10 integers. We can use realloc() to resize the memory block:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *ptr = (int *)malloc(5 * sizeof(int));
    // Resize the memory block to hold 10 integers
    ptr = (int *)realloc(ptr, 10 * sizeof(int));
    // Check for allocation failure
    if (ptr == NULL) {
        printf("Memory Reallocation Failed");
        exit(0);
    }
    return 0;
}
```



It is important to note that if realloc() fails and returns NULL, the original memory block is not freed, so you should not overwrite the original pointer until you've successfully allocated a new block. To prevent memory leaks, it's a good practice to handle the NULL return value carefully:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr = (int *)malloc(5 * sizeof(int));

    // Reallocation
    int *temp = (int *)realloc(ptr, 10 * sizeof(int));

    // Only update the pointer if reallocation is successful
    if (temp == NULL)
        printf("Memory Reallocation Failed\n");
    else
        ptr = temp;

    return 0;
}
```

Practical Example

- Consider the first scenario where we were having issues with the fixes size array. Let's see how we can resolve both of these issues using dynamic memory allocation.

```
#include <stdio.h>
#include <stdlib.h>

int main() {

    // Initially allocate memory for 5 integers
```

```
    int *ptr = (int *)malloc(5 * sizeof(int));

    // Check if allocation was successful
    if (ptr == NULL) {
        printf("Memory Allocation Failed\n");
        exit(0);
    }

    // Now, we need to store 8 elements so
    // Reallocate to store 8 integers
    ptr = (int *)realloc(ptr, 8 * sizeof(int));

    // Check if reallocation was successful
    if (ptr == NULL) {
        printf("Memory Reallocation Failed\n");
        exit(0);
    }

    // Assume we only use 5 elements now
    for (int i = 0; i < 5; i++) {
        ptr[i] = (i + 1) * 10;
    }

    // Shrink the array back to 5 elements
    ptr = (int *)realloc(ptr, 5 * sizeof(int));

    // Check if shrinking was successful
    if (ptr == NULL) {
        printf("Memory Reallocation Failed\n");
        exit(0);
    }

    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);

    // Finally, free the memory when done
    free(ptr);

    return 0;
}
Output
10 20 30 40 50
```

Explanation:

In this program, we are managing the memory allocated to the pointer ptr according to our needs by changing the size using realloc(). It can be a fun exercise to implement an array which grows according to the elements inserted in it. This kind of arrays are called dynamically growing arrays.

Issues Associated with Dynamic Memory Allocation

As useful as dynamic memory allocation is, it is also prone to errors that requires careful handling to avoid the high memory usage or even system crashes. Few of the common errors are given below:

- **Memory Leaks:** Failing to free dynamically allocated memory leads to memory leaks, exhausting system resources.
- **Dangling Pointers:** Using a pointer after freeing its memory can cause undefined behavior or crashes.
- **Fragmentation:** Repeated allocations and deallocations can fragment memory, causing inefficient use of heap space.
- Allocation Failures: If memory allocation fails, the program may crash unless the error is handled properly.

What is Memory Leak? How can we avoid?

- Data can be stored in either stack or heap memory. The stack stores local variables and parameters of the function while the heap is used for dynamic memory allocation during runtime.
  A memory leak occurs when a program dynamically allocates memory but does not release it after it's no longer needed.
- In C, memory is allocated using malloc() / calloc() and released using free().
- In C++, memory is allocated using new / new[] and released using delete / delete[].
- Note that even a small amount of leaked memory can cause problems in long running programs like servers which keep running.

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{

    // allocate memory
    int *ptr = (int *)malloc(sizeof(int));

    *ptr = 10;
    printf("%d\n", *ptr);

    // Forgot to free memory free(ptr) is missing
    return 0;
}
```

Common Causes of Memory Leak

- Memory is allocated (malloc/new) but not released (free/delete).
- If a pointer to allocated memory is overwritten or goes out of scope without freeing, the memory it pointed to becomes unreachable.

- In long-running programs, even small leaks add up over time. This gradually eats up system memory, making the program slow or crash.

How to avoid memory leaks?

- Always free/ delete the memory after use.
- Don't overwrite a pointer before freeing its old memory.
- In C++, prefer smart pointers (unique_ptr, shared_ptr) to handle memory automatically.
- Use tools like Valgrind to detect leaks during testing.

C PREPROCESSOR DIRECTIVES

- The C preprocessor is a program that processes the source code before actual compilation begins. It handles special instructions called preprocessor directives, which guide the compiler on how to prepare the code.
- Preprocessor directives start with the # symbol
- They are executed before compilation
- They are used for macros, file inclusion, and conditional compilation

Preprocessor Directives in C

#define

The #define directive is used to define macros or symbolic constants. These values are replaced before compilation.

- Used to define constants and macros
- Improves code readability
- No memory allocation happens

```
#include <stdio.h>

#define PI 3.14159

int main()
{
    // Using the PI macro to calculate
    double r = 8.0;
    10double area = PI * r * r;

    printf("%f\n", a);
    return 0;
}
Output
201.061760
```

#include

The #include directive is used to include header files in a program before compilation.

- Allows use of library functions
- Supports reusable code
- Comes in two forms: < > and " "

Syntax

```
#include <filename>
#include "filename"

< > -> system header files
" " -> user-defined header files

#include <stdio.h>

int main() {
    printf("Hello, Geek!\n");
    return 0;
}
Output
Hello, Geek!
```

Conditional Compilation Directives (#if, #elif, #else, #endif)

These directives work together to control which parts of the program get compiled based on certain conditions.

- If the condition after the #if is true, the lines after it will be compiled.
- If not, it checks the condition after associated #elif. If that's true, those lines will be compiled.
- If neither condition is true, the lines after #else will be compiled.

```
#include <stdio.h>

int main()
{
#define VALUE 2

// Check if VALUE is greater than 3
#if VALUE > 3
    printf("Value is greater than 3\n");

// Check if VALUE is 3
#elif VALUE == 3
    printf("Value is 3");

// If neither conditions are true, this
// block will execute
#else
    printf("Value is less than or equal to 2");
#endif

    return 0;
}
Output
```

```
Value is less than or equal to 2
Note: the entire structure of #if, #elif and #else chained directives ends with
#endif.
```

#ifdef

The #ifdef (if defined) directive checks whether a macro is defined. If it is, the code within the #ifdef block is included in the program.

```
#include <stdio.h>
int main()
{
#define DEBUG
// Check if DEBUG is defined
#ifdef DEBUG
    printf("Debugging is enabled");
#endif
    return 0;
}
Output
Debugging is enabled
```

#ifndef

The #ifndef (if not defined) directive checks if a macro is not defined. If it is not defined, the code inside the #ifndef block is included.

```
#include <stdio.h>
int main()
{

// Check if DEBUG is not defined
#ifndef DEBUG
    printf("Debugging is not enabled");
#endif
    return 0;
}
Output
Debugging is not enabled
```

#line

The #line directive is used to change line number and file name for the compiler.

- Useful in generated code
- Helps in debugging
- Alters compiler reporting

```c
#include <stdio.h>

// Define macro that prints current line number and filename
#define PrintLineNum printf("Line number is %d in file named %s\n", __LINE__,
__FILE__)

int main()
{
    PrintLineNum;

    // Change line number to 20 and filename to "main.c"
    PrintLineNum;

// Change line number to 30 and filename to "index.c"
#line 30 "index.c"
    PrintLineNum;

    return 0;
}
Output
Line number is 5 in file named ./Solution.c
Line number is 20 in file named main.c
Line number is 30 in file named index.c
```

Line number that will be assigned to the next code line. The line numbers of successive lines will be increased one by one from this point on. "filename" - optional parameter that allows to redefine the file name that will be shown.

#error

The #error directive generates a compilation error with a custom message.

- tops compilation immediately
- used for enforcing conditions

```c
#include <stdio.h>

#ifndef GeeksforGeeks

// Show error message
#error "GeeksforGeeks not found!"
#endif

int main()
{
    printf("Hello, GeeksforGeeks!\n");
    return 0;
}

Output
```

```
error: #error "GeeksforGeeks not found!"
```

#pragma - Pragma Directive

The #pragma directive gives special instructions to the compiler.

- Compiler-specific behavior
- Used for optimizations and warnings

Commonly used pragma directives are:

- #pragma message: used at compile time to print custom messages.
- #pragma once: to guard the header files i.e the header file must be included only once.
- #pragma warning: to enable or disable the warnings.

```
   #include <stdio.h>


// not defining gfg to trigger pragma message
// #define GeeksforGeeks


   int main()

   {

   #ifndef GeeksforGeeks

   #pragma message(" GfG is not defined.")

   #endif

       printf("Hello geek!\n");

       return 0;

   }
```

Output

```
#pragma message:  GfG is not defined.
```

Summary of Common Preprocessor Directives in C

Preprocessor Directives	Description
#define	Used to define a macro.
#undef	Used to undefine a macro.
#include	Used to include a file in the source code program.
#ifdef	Used to include a section of code if a certain macro is defined by #define.
#ifndef	Used to include a section of code if a certain macro is not defined by #define.
#if	Check for the specified condition.
#else	Alternate code that executes when #if fails.
#endif	Used to mark the end of #if, #ifdef, and #ifndef.
#error	Used to generate a compilation error message.
#line	Used to modify line number and filename information.
#pragma once	To make sure the header is included only once.
#pragma message	Used for displaying a message during compilation.

#pragma Directive in C

- In C, the #pragma directive is a special purpose directive that is used to turn on or off some features. #pragma also allows us to provide some additional information or instructions to the compiler. It is compiler-specific i.e., the behavior of pragma directive varies from compiler to compiler.

Syntax of #pragma

```
#pragma directive_name
```

Commonly Used #pragma Directives in C

Some of the commonly used #pragma directives are discussed below:

1. #pragma startup and #pragma exit

- These directives help us to specify the functions that are needed to run before the program starts (before the control passes to the main()) and just before the program exits (just before the control returns from the main()).

Syntax

```
#pragma startup function_name
#pragma exit function_name
```

Example

The below program will not work with GCC compilers. Look at the below program:

```
// C program to demonstrate the working of #pragma startup
// and #pragma exit

#include <stdio.h>

void func1();
void func2();

#pragma startup func1
#pragma exit func2

void func1() { printf("Inside func1()\n"); }

void func2() { printf("Inside func2()\n"); }

int main()
{
    printf("Inside main()\n");

    return 0;
}
Output
Inside func1()
Inside main()
Inside func2()
```

The above code will produce the output as given below when run on GCC compilers:

Inside main()

This happens because GCC does not support **#pragma startup or exit**. However, you can use the below code for a similar output on GCC compilers.

```
// C program to demonstrate the working of #pragma startup
// and #pragma exit (without GCC compiler)


#include <stdio.h>


void func1();

void func2();
```

```
void __attribute__((constructor)) func1();

void __attribute__((destructor)) func2();


void func1() { printf("Inside func1()\n"); }


void func2() { printf("Inside func2()\n"); }


int main()

{

    printf("Inside main()\n");

    return 0;

}
```

Output

```
Inside func1()

Inside main()

Inside func2()
```

2. #pragma warn Directive

- This directive is used to hide the warning messages which are displayed during compilation. This may be useful for us hen we have a large program and we want to solve all the errors before looking on warnings then by using it we can focus on errors by hiding all warnings. we can again let the warnings be visible by making slight changes in syntax.

Syntax

```
#pragma warn +xxx (To show the warning)
#pragma warn -xxx (To hide the warning)
#pragma warn .xxx (To toggle between hide and show)
```

We can hide the warnings as shown below:

- **#pragma warn -rvl**: This directive hides those warning which are raised when a function which is supposed to return a value does not return a value.
- **#pragma warn -par**: This directive hides those warning which are raised when a function does not uses the parameters passed to it.
- **#pragma warn -rch**: This directive hides those warning which are raised when a code is unreachable. For example: any code written after the return statement in a function is unreachable.

Example

The below example demonstrates the use of above warnings.

```
// C program to explain the working of
// #pragma warn directive
// This program is compatible with C/C++ compiler

#include<stdio.h>

#pragma warn -rvl /* return value */
#pragma warn -par /* parameter never used */
#pragma warn -rch /*unreachable code */

int show(int x)
{
    // parameter x is never used in
    // the function

    printf("GEEKSFORGEEKS");

    // function does not have a
    // return statement
}

int main()
{
    show(10);

    return 0;
}

Output
GEEKSFORGEEKS
The above program compiles successfully without any warnings to give the output
"GEEKSFORGEEKS".
```

3. #pragma GCC poison

- This directive is supported by the GCC compiler and is used to remove an identifier completely from the program. If we want to block an identifier then we can use the #pragma GCC poison directive.

Syntax

```
#pragma GCC poison identifier
```

Example

The below example demonstrates the use of #pragma GCC poison.

```
// C Program to illustrate the
// #pragma GCC poison directive

#include <stdio.h>

#pragma GCC poison printf

int main()
{
    int a = 10;

    if (a == 10) {
        printf("GEEKSFORGEEKS");
    }
    else
        printf("bye");

    return 0;
}
Output
prog.c: In function 'main':
prog.c:14:9: error: attempt to use poisoned "printf"
         printf("GEEKSFORGEEKS");
         ^
prog.c:17:9: error: attempt to use poisoned "printf"
         printf("bye");
         ^
prog.c: In function 'main':
prog.c:14:9: error: attempt to use poisoned "printf"
         printf("GEEKSFORGEEKS");
         ^
prog.c:17:9: error: attempt to use poisoned "printf"
         printf("bye");
         ^
```

4. #pragma GCC dependency

- The #pragma GCC dependency allows you to check the relative dates of the current file and another file. If the other file is more recent than the current file, a warning is issued. This is useful if the current file is derived from the other file, and should be regenerated.

Syntax

```
#pragma GCC dependency "main.c"
```

Example

In the below program #pragma GCC dependency "parse.y" is used to indicate that this file depends on the file named "parse.y"

```
// C program to demonstrate use of

#include <stdio.h>

// Using #pragma GCC dependency to check if "parse.y" file
// has been modified
#pragma GCC dependency "solution.c"

int main()
{
    printf("Hello, Geek!");
    return 0;
}
Explanation:
At the compile time compiler will check whether the "parse.y" file is more recent
than the current source file. If it is, a warning will be issued during
compilation.
```

5. #pragma GCC system_header

- The `#pragma GCC system_header` directive in GCC is used to tell the compiler that the given file should be treated as if it is a system header file. warnings that will be generated by the compiler for code in that file will be suppressed hence it is extremely useful when the programmer doesn't want to see warnings or errors. This pragma takes no arguments. It causes the rest of the code in the current file to be treated as if it came from a system header.

Syntax

```
#pragma GCC system_header
```

Example

The below example demonstrates the use of #pragma GCC system_header.

```
// external_library.h
#ifndef EXTERNAL_LIBRARY_H
#define EXTERNAL_LIBRARY_H

#pragma GCC system_header
void externalFunction();
#endif
```

The above code is for external library that we want to include in our main code. So to inform the compiler that external_library.h is a system header, and we don't want to see warnings from it the #pragma GCC system_header is used.

```
// C program to demonstrate #pragma GCC system_header
#include <stdio.h>
#include "external_library.h"

int main() {
    externalFunction();  // Using a function from the external library

    printf("Hello, Geek!\n");
    return 0;
}
Output
Hello, Geek!
Explanation:
Now inour main program,"external_library.h" header is included and the function
declared in it is used. Hello,Geek ! is printed without any warnings.
```

6. #pragma once

- The #pragma once directive has a very simple concept. The header file containing this directive is included only once even if the programmer includes it multiple times during a compilation. This directive works similar to the #include guard idiom. Use of #pragma once saves the program from multiple inclusion optimisation.

Syntax

```
#pragma once
```

Example

The below example demonstrates the use of #pragma once.

```
// my_header.h
#pragma once

void myFunction();
```

Consider the above my_header.h header file in this header file, #pragma once is used to guard against multiple inclusions.

```
// C program to demonstrate the use of pragma once
#include <stdio.h>
#include "my_header.h"
```

```
int main() {
    myFunction();  // Using a function from my_header.h

    printf("Hello, World!\n");
    return 0;
}
Output
Hello, World!
```

MACROS AND ITS TYPES IN C

- In C programming, a macro is a symbolic name or constant that represents a value, expression, or code snippet. They are defined using the #define directive, and when encountered, the preprocessor substitutes it with its defined content.

Example

```
#include <stdio.h>

// Macro definition
#define LIMIT 5

int main(){

    // Print the value of macro defined
    printf("LIMIT: %d", LIMIT);

    return 0;
}
Output
LIMIT: 5
Explanation:
In this code, a macro LIMIT is defined with the value 5 using the #define
directive. The macro LIMIT is then used in the printf function to print its value,
which is 5. The preprocessor replaces the macro LIMIT with its defined value
before the code is compiled, so the output of the program will display the value
5.
```

Syntax

The general syntax to define a macro is:

```
#define MACRO_NAME value
```

where **MACRO_NAME** is the name of the macro and **value** is the code or value that replaces the macro name. It is to be noted that macros don't necessarily need to be in uppercase. Instead, it is a convention that makes it easy to recognize.

Types Of Macros in C

There are two types of macros in C language:

1. Object-Like Macros

- Object-like macros are the simplest type of macros. They replace the macro name with a defined value or expression. These are used for constants or simple values.

```
#include <stdio.h>

// Macro definition
#define DATE 31

int main(){

    // Print the message
    printf("Lockdown will be extended"
           " upto %d-MAY-2020",
           DATE);
    return 0;
}
Output
Lockdown will be extended upto 31-MAY-2020
Explanation:
In this program, the object-like macro DATE is defined as 31. It is used in the
printf function to insert the value 31 into the message, which the preprocessor
replaces before compilation.
```

2. Chain Macros

- These macros involve chaining multiple macros together. This can be done by combining different macros in a single macro definition, allowing for more complex operations. In chain macros first of all parent macro is expanded then the child macro is expanded.

```c
#include <stdio.h>

// Macro definition
#define INSTAGRAM FOLLOWERS
#define FOLLOWERS 138

int main(){
    printf("Geeks for Geeks have %dK"
           " followers on Instagram",
           INSTAGRAM);

    return 0;
}
```

Output

```
Geeks for Geeks have 138K followers on Instagram
```

Explanation:

INSTAGRAM is expanded first to produce **FOLLOWERS**. Then the expanded macro is expanded to produce the outcome as **138K.** This is called the chaining of macros.

3. Multi-Line Macros

These macros span multiple lines for readability and organization. They are often used when you need a more complex expression or code block. To create a multi-line macro you have to use **backslash \.**

```c
#include <stdio.h>

// Multi-line Macro definition
#define ELE 1, \
            2, \
            3

int main(){

    // Array arr[] with elements
    // defined in macros
    int arr[] = { ELE };
    for (int i = 0; i < 3; i++) {
        printf("%d  ", arr[i]);
    }
    return 0;
}
```

Output

```
1  2  3
```

Explanation:

In this program, a multi-line macro **ELE** is defined with the values **1, 2, 3**, which is used to initialize the array **arr[]**. The preprocessor replaces **ELE** with these values.

4. Function-Like Macros

- These macros take parameters and behave like functions, allowing you to define reusable logic for common operations. They are expanded at compile time. A function-like macro is only lengthened if and only if its name appears with a pair of parentheses after it. If we don't do this, the function pointer will get the address of the real function and lead to a syntax error.

```
#include <stdio.h>

// Function-like Macro definition
#define min(a, b) (((a) < (b)) ? (a) : (b))

int main(){

    // Given two number a and b
    int a = 18, b = 76;

    printf("Minimum: %d", min(a, b));

    return 0;
}
Output
Minimum: 18

Explanation:
In this code, the function-like macro min(a, b) compares two values and returns the smaller using the ternary operator. The values 18 and 76 are passed to the macro, which returns 18.
```